



Trackvance Core

Especificación técnica v1.1

IMPLEMENTACIÓN 0.5.1 · EVOLUCIÓN FUNCIONAL LOCAL

25 septiembre 2026 · Estado funcional y evidencia de validación

Arquitectura local y evolución a producto

Monolito modular · Persistencia local · Reglas portables · Evidencia verificable

Documento oficial de referencia. Describe el estado real de Trackvance Core y distingue las capacidades implementadas de los contratos preparados y la infraestructura objetivo.

Trackvance Colombia SAS

Contenido

1. Alcance, estados e invariantes6

Invariantes compartidos6

Qué no se certifica como implementado6

2. Arquitectura lógica estable7

Separación incremental8

3. Despliegue local de referencia8

4. Código, stack y dependencias9

Responsabilidades del backend10

5. Fuentes y lectores de datasets11

Límites locales11

Conectores externos11

5A. Módulo Conexiones12

Límites de responsabilidad12

Representación y límites12

Versiones y evidencia12

SecretStore, permisos y auditoría12

Fuentes futuras y frontera de salida13

5B. Data Delivery13

A. Propósito y flujo funcional13

B. Identidad y versiones del destino13

C. Contrato DataSink y límites de responsabilidad14

D. Mapping, selección y orden14

E. Texto y temporales: ejemplos exactos14

F. Target nuevo o existente15

G. CREATE_AND_LOAD15

H. APPEND15

I. OVERWRITE16

J. UPSERT y claves compuestas16

K. PostgreSQL: transacción y conteos fiables16

L. SQL Server: concurrencia y conteos17

M. Familias de preflight y fallos17

N. Preview, configuración y publicación17

O. Workers, lanes y frontera de commit17

P. Estados del intento y del Run18

Q. UNKNOWN: revisión operacional append-only 18

R. COMMITTED + PENDING_REPAIR: sólo evidencia local 19

S. Idempotencia y concurrencia 19

T. Métricas: significado y límites 19

U. Receipt y manifest: ejemplo abreviado 20

V. Linaje persistido y auditoría 20

W. Permisos, secretos y aislamiento 21

X. Backup, restauración y compatibilidad 21

Y. Benchmark dedicado y operación en UI 21

Z. Límites y criterio de aceptación 22

6. Carga, esquema y versionado 23

 Secuencia persistente 23

 Caso operativo: snapshot, refresh e historial 23

7. Profiling, identificadores y transforms 24

 Inferencia inequívoca 24

 Transformaciones funcionales 24

8. Reglas portables y Data Intake 25

 Condiciones y referencias seguras 25

 Editor y resultados 26

 Caso operativo: aceptación y evidencia reutilizable 26

9. ReconOps y normalización declarada 26

 Políticas y orden de ejecución 27

 Clasificaciones estables 27

 Caso operativo: claves, tolerancia y diferencias 27

10. Sentinel y métricas históricas 28

 Compatibilidad de series 28

 Programación local e historia visible 28

 Caso operativo: monitoreo y comparación histórica 29

11. Excepciones con validación técnica 29

 Criterios de elegibilidad 30

 Gestión operativa 30

 Caso operativo: cierre respaldado por nueva ejecución 30

12. Centro de Control y navegación 31

 Rutas SPA 31

 Carga diferida y continuidad de la navegación 31

13. StorageProvider, artefactos y linaje	32
14. Manifest schema 2 y auditoría	33
Identidad y eventos	33
15. Reportes Excel de negocio	34
Tipos y seguridad	34
16. Ejecución, planificación y recuperación	34
ExecutionPlanner	35
17. Modelo persistente y migraciones	35
Cadena de trazabilidad	36
18. API y errores	37
19. Contratos HTTP y permisos	40
Administración local implementada	40
Errores operativos	40
20. Configuración, operación y respaldo	41
Respaldo Docker consistente	42
Restauración en proyecto nuevo	42
Reset deliberado	43
20A. Benchmark reproducible y límites comprobados	43
Antecedente de volumen 0.4.0; no certifica 0.5.1	43
Representatividad y progresión detenida	44
21. Arquitectura final de producto	44
Pasos de evolución sin cambiar reglas	45
22. Calidad, pruebas y aceptación	45
Certificación incremental 0.5.1	46
Antecedente certificado 0.5.0 (no certifica 0.5.1)	46
Base certificada de 0.4.1	47
Base certificada de 0.4.0	47
Escenarios de negocio conservados	47
Incidencia CI identificada y corrección	48
Pendientes explícitos	48
23. Decisiones vigentes y control documental	49
Precedencia y cambios de esta revisión	49
Registro	50

24. Cambios funcionales y técnicos 0.5.1	50
1. Reparación local de evidencia COMMITTED	50
2. Revisión operacional de UNKNOWN	51
3. Semántica uniforme de métricas	51
4. Desglose UPSERT PostgreSQL por capacidades	51
5. Regresión temporal exhaustiva	51
6. Linaje canónico sin migración histórica	51
7. Benchmark exclusivo de Delivery	52
8. División del frontend en módulos de carga diferida	52

1. Alcance, estados e invariantes

Trackvance Core es una plataforma local de confiabilidad de datos. Integra Data Intake, ReconOps, Sentinel y Data Delivery con Datasets, Excepciones, Centro de Control, Auditoría e Identidad/RBAC. El usuario carga o versiona datos, publica una configuración inmutable, ejecuta controles o entregas y conserva evidencia verificable del resultado.

Esta especificación describe el endurecimiento local 0.5.1 sobre la baseline 0.5.0. Conserva Data Delivery PostgreSQL/SQL Server y añade reparación de evidencia, revisión operacional UNKNOWN, métricas honestas, conteos UPSERT fiables cuando el motor los proporciona, regresión temporal, linaje preciso, benchmark dedicado y carga diferida del frontend. El documento conserva su nombre v1.1. La tabla de validación informa cada ejecución real y distingue antecedentes, pendientes y limitaciones; no se heredan éxitos de otra versión.

Estado	Significado normativo
IMPLEMENTADO	Existe en el código local y en el flujo operativo, con cobertura automatizada o evidencia ejecutada.
PREPARADO	Existe un puerto, contrato versionado o rechazo controlado. El adaptador productivo todavía requiere implementación y validación.
OBJETIVO	Forma parte de la arquitectura final prevista; no es una dependencia ni capacidad operativa de la instalación local.

Invariantes compartidos

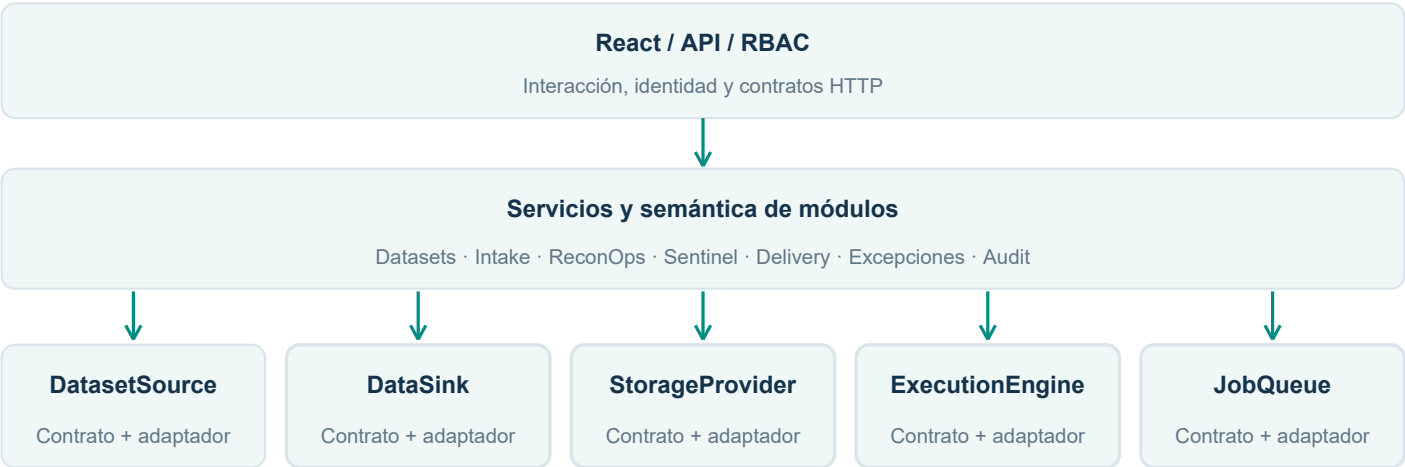
- La arquitectura lógica es un monolito modular. API y workers pueden desplegarse como procesos separados del mismo producto; no son microservicios de negocio.
- PostgreSQL guarda metadata y relaciones. Los archivos recibidos, Parquet canónicos, resultados, manifiestos y exports se guardan fuera de PostgreSQL.
- DatasetVersion, configuración publicada y Run conservan identidades estables. Una modificación funcional crea una versión o ejecución nueva.
- SUCCESS significa que terminó el procesamiento. REJECTED, WITH_FINDINGS y ALERT son decisiones de negocio y no errores técnicos.
- En Data Delivery, COMMITTED exige confirmación remota. UNKNOWN preserva una confirmación incierta; no equivale a FAILED ni COMMITTED y no habilita reintento automático.
- Profiling observa los valores recibidos sin trim, case o normalización Unicode funcional implícita. Las transformaciones de negocio deben estar declaradas.
- Toda modificación del esquema persistente se realiza mediante una migración Alembic nueva. No se editan migraciones ya aplicadas.
- La autorización por rol, permiso y organización se verifica en backend; ocultar un botón no sustituye la autorización.
- El prototipo funciona localmente con imágenes preparadas. No necesita servicios cloud para operar.

Qué no se certifica como implementado

PySpark operativo, Redis/Celery productivo, fuentes o destinos distintos de PostgreSQL/SQL Server, object storage remoto, OIDC/SSO, Kubernetes, Terraform, Helm, observabilidad distribuida, masking por sensibilidad y escalamiento de producción pertenecen a los estados PREPARADO u OBJETIVO. Data Delivery SQL está implementado, pero no incluye scheduler, transforms de negocio, 2PC, reintento ciego tras UNKNOWN ni RBAC granular propio. El roadmap completa los puntos funcionales 1 a 10; la productización del punto 11 requiere una evolución posterior.

2. Arquitectura lógica estable

La separación principal es entre adquisición de datos, almacenamiento interno y ejecución. Estos conceptos tienen responsabilidades y credenciales distintas. Leer un archivo de un bucket del cliente no implica almacenar los artefactos internos en ese mismo bucket.



Puerto / componente	Responsabilidad	Adaptador local
DataSource	Adquirir datos desde una fuente y devolver el modelo común.	Archivos locales, PostgreSQL y SQL Server
DataSink	Descubrir, validar y escribir una DatasetVersion en un destino externo.	PostgreSQL y SQL Server
SecretStore	Gestionar credenciales por organización/recurso, fuera de artifacts.	Stores y claves Fernet separados para fuentes y destinos
DatasetReader	Interpretar un formato de archivo y sus opciones de lectura.	CSV, XLSX, JSON, Parquet, TXT
StorageProvider	Publicar bytes inmutables, materializar un artefacto y asignar staging temporal.	FileArtifactStore
ExecutionEngine	Ejecutar un Run completo con evidencia y linaje.	LocalExecutionEngine
ProcessingEngine	Compilar/evaluar expresiones portables de reglas y comparaciones.	Polars; DuckDB para paridad cubierta
ExecutionPlanner	Estimar recursos y fijar un plan antes de ejecutar.	Presupuesto de memoria/disco local
JobQueue	Publicar el trabajo asociado a un Run.	DatabaseJobQueue
NotificationDelivery	Contrato para entregar una alerta fuera del sistema.	PREPARADO; las alertas actuales son Findings internos

Las reglas no reciben cadenas de conexión ni conocen CSV, Excel o buckets. Trabajan sobre el modelo común producido por los lectores y el Parquet canónico. La orquestación usa identidades de versiones y configuraciones, genera hallazgos y publica evidencia mediante los puertos.

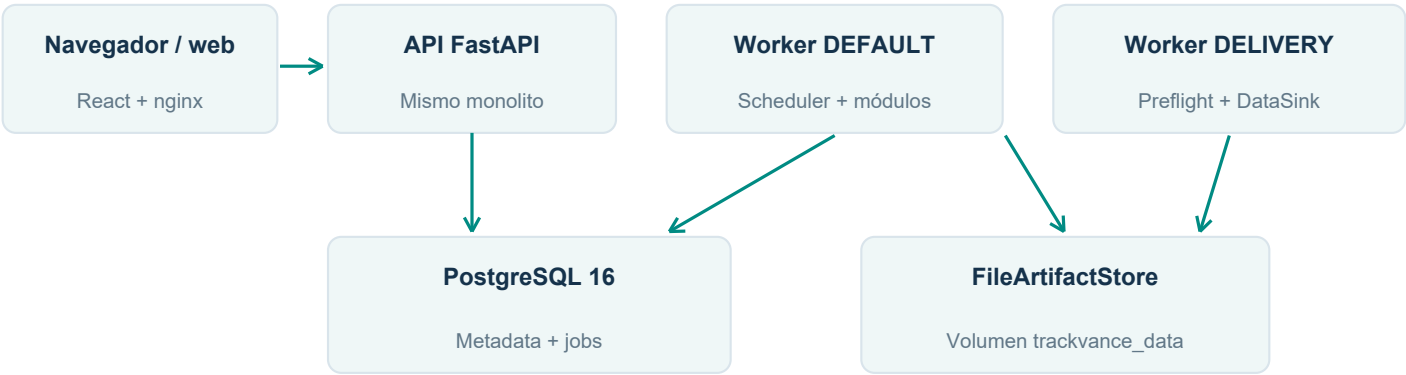
Separación incremental

La implementación mantiene un paquete Python compacto. API, servicios, modelos ORM, reglas, adapters y worker están en módulos distintos, pero aún no existe una capa de dominio totalmente libre de ORM. ExecutionEngine y JobQueue reciben Session/Run de SQLAlchemy. Esta frontera es suficiente para sustituir infraestructura de forma progresiva; no se declara una arquitectura hexagonal pura ni una sustitución cloud inmediata sin trabajo adicional.

StorageProvider devuelve Path únicamente para staging/materialización necesarios por los motores locales. Los servicios no deciden el directorio del volumen. Los campos históricos *_path permanecen compatibles como referencias internas; una migración futura de locators debe preservar IDs, hashes y linaje.

3. Despliegue local de referencia

IMPLEMENTADO. Docker Compose es el entorno de referencia para operación y certificación con PostgreSQL. El arranque directo Windows/SQLite se conserva como comodidad de desarrollo y usa una instalación de datos independiente.



Servicio	Función	Persistencia / red
web	React compilado servido por nginx y proxy a /api.	Publicado solo en 127.0.0.1; puerto configurable.
api	FastAPI, sesión/RBAC, carga, configuración, consulta y downloads.	Red interna Compose; comparte trackvance_data.
worker	Lane DEFAULT: despacha programación Sentinel y procesa Intake/ReconOps/Sentinel, leases y heartbeat.	Red interna; comparte metadata y trackvance_data; no monta secretos.
delivery-worker	Lane DELIVERY: ejecuta preflight final y transacciones Data Delivery; no despacha scheduler.	Red interna; comparte metadata, trackvance_data y secretos de destino.
postgres	PostgreSQL 16 para metadata.	Volumen postgres_data; sin puerto publicado al host.

El volumen trackvance_data se monta en /var/lib/trackvance para API y ambos workers. FileArtifactStore administra artifacts/<id>/data.<ext> y staging tmp; las rutas históricas permanecen legibles. No se guardan datasets como blobs SQL. Los volúmenes sobreviven a stop, restart, recreación de contenedores y docker compose down. down -v elimina volúmenes y no es un comando de actualización.

API monta connection_credentials/connection_keys para fuentes y delivery_credentials/delivery_keys para destinos. El worker DEFAULT monta únicamente trackvance_data. El delivery-worker monta trackvance_data y los dos volúmenes de destino, nunca los de fuente. La copia de recuperación debe conservar metadata, artifacts y ambos pares secreto/clave de forma coordinada.

Todos los servicios declaran restart: "no". Trackvance permanece detenido al iniciar Docker Desktop hasta que el usuario ejecute el arranque. La instancia del equipo usa trackvance-certification y http://localhost:3100; el valor por defecto del repositorio es 3000.

```
docker compose up --build -d --wait
docker compose stop
docker compose up -d --wait
docker compose ps
```

La preparación descarga imágenes y dependencias. Una vez construidas, los flujos sobre snapshots locales no requieren storage, identidad ni procesamiento cloud. Conexiones y Data Delivery sí requieren red hacia la fuente/destino seleccionado. El override de red interna bloquea esas integraciones externas; la salida de nginx depende de Docker Desktop/firewall y no se presenta como aislamiento total del host.

4. Código, stack y dependencias

Capa	Implementación actual
Frontend	React 19, TypeScript, Vite, React Router, TanStack Query, Radix Dialog, Lucide y CSS propio.
HTTP / persistencia	FastAPI, Pydantic, SQLAlchemy 2, Alembic, psycopg y PostgreSQL 16.
Procesamiento	Polars/Python, Decimal, expresiones portables y compilador DuckDB.
Formatos / evidencia	openpyxl para XLSX, lectores locales, Parquet, JSON y SHA-256.
Pruebas	pytest, Ruff, Mypy, ESLint, TypeScript, Vitest, Testing Library y Playwright.
Reproducibilidad	backend/uv.lock, frontend/pnpm-lock.yaml, Dockerfiles y compose.yml.

Responsabilidades del backend

Archivo	Responsabilidad principal
api.py	DTOs HTTP, sesiones, autorización, errores y endpoints.
services.py	Casos de uso: versiones, runs, evidencia, hallazgos y validación de excepciones.
models.py / db.py	Modelo persistente y unidad de sesión SQLAlchemy.
processing.py / config_semantics.py	Semántica de perfil, reglas, normalización y resultados.
portable_engine.py / planner.py	Compilación portable, evaluación y preflight.
dataset_readers.py	DatasetSource, registry y lectores por formato.
connections_api.py / connections_service.py	Contratos HTTP, configuración versionada, bindings y adquisición de fuentes.
dataset_sources.py / credential_store.py	Adaptadores SQL de solo lectura y contrato/proveedor de secretos.
delivery_api.py / delivery_schemas.py	Contratos HTTP de destinos, preview, preflight, configuraciones, runs, intentos y receipt.
delivery_service.py / data_sinks.py / delivery_credential_store.py	Orquestación, adaptadores SQL de escritura, transacciones, evidencia y secretos de destino.
artifactstore.py	StorageProvider y adaptador local inmutable.
execution.py / jobqueue.py / worker.py	Puerto de ejecución, publicación, consumo y recuperación local.
manifests.py / exports.py	Compatibilidad de evidencia y reportes Excel.
permissions.py / audit_context.py	Política RBAC, actor y sanitización de metadata.
dashboard.py	Agregación operativa del Centro de Control.
scheduler.py / sentinel_api.py	Programaciones versionadas, ocurrencias, despacho local e histórico de series.
exceptions_api.py	Asignación, SLA, comentarios, adjuntos y gestión de casos con concurrencia optimista.
identity_api.py	Usuarios locales, roles, permisos efectivos, contraseñas y revocación de sesiones.
operations_common.py	Inventario e integridad compartidos por las herramientas de mantenimiento.

El frontend se organiza en app, components, features/datasets, features/connections, features/delivery, features/runs, features/identity y routes; comparte cliente HTTP, componentes visuales y queries. RuleBuilder, ComparisonBuilder y TransformBuilder utilizan esquema real; MonitorSchedulePanel gestiona programación e historia; Delivery guía destino, mapping, preview, preflight, publicación y evidencia. La organización por bounded contexts/domain/application/infrastructure podrá profundizarse cuando el tamaño lo justifique.

No forman parte del stack instalado React Hook Form, Zod, Tailwind, TanStack Table, Recharts, PyArrow ni PySpark. La documentación inicial que los proponía se interpreta como antecedente de diseño, no como inventario actual.

5. Fuentes y lectores de datasets

IMPLEMENTADO. `DatasetSource.read(options, inspect)` devuelve `DatasetReadResult`. `LocalFileDatasetSource` resuelve el lector en `DatasetReaderRegistry`. Cada lector interpreta el formato sin aplicar transformaciones de negocio.

`DatasetReadResult` contiene `frame`, `source_format`, `format_label`, `media_type`, `row_numbering`, `native_schema`, hojas/opción seleccionada, delimitador detectado, `row_count` y metadata acotada. El frame común representa escalares observados como `String/Null`. La metadata nativa informa la inferencia sin alterar la representación recibida.

Fuente local	Detección y comportamiento
CSV	UTF-8 con BOM opcional; coma/punto y coma conservan compatibilidad. Conserva líneas físicas, incluidos registros multilínea.
XLSX	Verifica paquete Excel, limita expansión y rechaza macros. Enumera hojas, permite seleccionar y usa lectura acotada. Las fórmulas se tratan como texto observado.
JSON	Arreglo de objetos, objeto individual, objeto con un único arreglo tabular y JSONL/NDJSON. Objetos anidados simples se aplanan con rutas de punto; listas internas permanecen JSON compacto.
Parquet	Usa esquema embebido y footer. Revisa filas, columnas y expansión antes de leer; la inspección no requiere decodificar todo el archivo.
TXT / TSV	Detección de coma, punto y coma, tabulación o barra vertical; permite fijar un carácter ASCII seguro.

La resolución usa firmas fuertes, contenido, extensión y detección delimitada cuando procede. MIME y extensión de descarga se derivan del formato verificado; un nombre engañoso no determina por sí solo el parser ni el tipo de descarga.

Límites locales

La carga admite por defecto 10 MiB, 100.000 filas y 100 columnas. La inspección previa usa hasta 100 registros y metadata embebida donde existe. Los lectores limitan expansiones XLSX/Parquet y tamaño de celdas. La inspección controla la muestra; la carga/perfil inicial completos siguen siendo síncronos y acotados.

Conectores externos

`PostgreSQLDatasetSource` y `SQLServerDatasetSource` están implementados. Resuelven credenciales mediante `SecretStore`, limitan consultas en origen, conservan metadata reproducible sin secretos y producen el modelo común. S3, Azure Blob, REST, Oracle, MySQL, Snowflake y Databricks son futuros adaptadores. La adquisición podrá ejecutarse en un job sin cambiar las reglas.

5A. Módulo Conexiones

IMPLEMENTADO. Permite crear, probar, editar, deshabilitar y eliminar lógicamente conexiones PostgreSQL y SQL Server. La conexión registra nombre, motor, host, puerto, base, usuario y opciones controladas. La contraseña se envía únicamente al guardar/probar y nunca vuelve en una respuesta. El formulario exige una prueba vigente y backend vuelve a verificar antes de guardar. La deshabilitación puede hacerse aunque la fuente esté caída.

Después de conectar, el usuario explora schemas, tablas y vistas accesibles. La vista previa muestra nombres, tipos nativos/lógicos, nulabilidad y hasta 100 registros. La detección del esquema usa catálogos; no escanea la tabla completa. Registrar la selección crea un Dataset, su binding de origen y una DatasetVersion canónica Parquet mediante StorageProvider. Los motores de calidad siguen leyendo versiones locales inmutables.

Límites de responsabilidad

El PostgreSQL interno guarda solo metadata del sistema. PostgreSQL/SQL Server externos son fuentes del usuario y tienen credenciales independientes. Ningún adaptador escribe filas en la fuente. No se recibe SQL arbitrario; se seleccionan identificadores previamente descubiertos y citados según el motor. PostgreSQL usa transacciones read-only. En SQL Server la intención read-only es informativa: es obligatorio usar una cuenta con permisos SELECT mínimos, sin permisos de escritura.

Las fuentes externas requieren acceso de red desde API. El overlay offline restringe esa salida; el Compose base permite conexión a fuentes accesibles, sin añadir dependencias cloud. Para fuentes en el PC bajo Docker Desktop puede usarse host.docker.internal. La configuración local conserva restart=no.

Representación y límites

El preview aplica LIMIT/TOP en origen; el snapshot lee por lotes de 100 hasta 100.000 filas, 100 columnas y 64 MiB de valores normalizados. Cada celda tiene límite de 64 KiB. Una fila adicional detecta el exceso y se rechaza la importación completa: nunca se guarda una muestra parcial como si fuera un dataset completo. Timeout configurable de conexión 1..15 segundos; consulta 1..60 segundos. La adquisición sigue siendo síncrona y acotada.

Los valores se conservan como texto/null: Decimal sin redondeo float, fechas ISO, Unicode y espacios originales. Los identificadores siguen la política id/*_id y permiten override. Los binarios son Base64. SQL Server timestamp se interpreta como rowversion; datetimeoffset (hasta escala 6) y PostgreSQL timestampz conservan TIMESTAMP con offset. timestamp without time zone, datetime, datetime2 y smalldatetime se adquieren como STRING: no se inventa una zona horaria y Delivery conserva el texto exacto. datetimeoffset(7) también se adquiere como STRING para conservar el séptimo dígito. El orden DATABASE_UNSPECIFIED depende del motor; SNAPSHOT_ROW es posición en el snapshot, no un número físico en la tabla externa.

Versiones y evidencia

La conexión tiene ID estable y revisión para concurrencia optimista. Cada edición crea ExternalConnectionVersion inmutable con hash de configuración y referencia opaca de credencial. Un binding conserva dataset, conexión, schema, tabla/vista y overrides. Actualizar desde fuente usa la configuración vigente y crea una versión nueva. Fallos de acceso no modifican versiones anteriores.

ingestion_metadata.source conserva connection_id, connection_version_id, connection_version, source_type, schema_name, object_name, object_kind, config_hash y captured_at. El linaje SOURCE_SNAPSHOT apunta a la revisión de conexión; REFRESH_OF apunta al snapshot anterior. Los manifests de runs incluyen la identidad de fuente de cada DatasetVersion. El run original y su hash permanecen inmutables aunque cambie la conexión.

SecretStore, permisos y auditoría

El contrato put/get/delete permite sustituir el proveedor local por Azure Key Vault, AWS Secrets Manager o HashiCorp Vault. La implementación local cifra con Fernet autenticado, vincula credencial a organización/referencia y persiste en connection_credentials; la clave maestra reside en connection_keys. En Linux los directorios son 0700 y archivos 0600; Windows directo hereda ACL del usuario. El backup debe proteger ambos volúmenes por separado del almacenamiento de artifacts. Una clave perdida no se reemplaza silenciosamente si ya existen secretos. Rotación automática de clave maestra permanece pendiente.

La contraseña guardada solo se reutiliza en pruebas o ediciones cuando host, puerto, base, usuario y modo TLS coinciden con la versión vigente. Cambiar cualquiera exige una contraseña explícita; su ausencia devuelve 422

PASSWORD_REQUIRED_FOR_ENDPOINT_CHANGE antes de contactar el destino. Nombre y timeouts pueden cambiar conservando el secreto. Esto impide redirigir una credencial guardada a otro servidor desde un formulario de edición.

Transporte cifrado por defecto: PostgreSQL sslmode=require y SQL Server encryption=require. PostgreSQL permite verify-ca/verify-full con CA configurada. Requerir cifrado no equivale a verificar identidad del servidor; la confianza de certificados debe administrarse en el driver. Los modos disable/off se reservan para pruebas locales. FreeTDS comparte timeouts a nivel proceso; el adaptador SQL Server serializa conexiones para evitar carreras.

RBAC separa connections:read, connections:use y connections:manage. Administradores y responsables de datos administran; Analista puede usar. Los roles de consulta solo ven metadata. Todas las rutas verifican organización. La auditoría registra creación, edición, prueba, preview, baja y adquisición con actor estable; nunca password, DSN ni filas de preview. Los errores de drivers se convierten a códigos controlados y mensajes constantes.

Fuentes futuras y frontera de salida

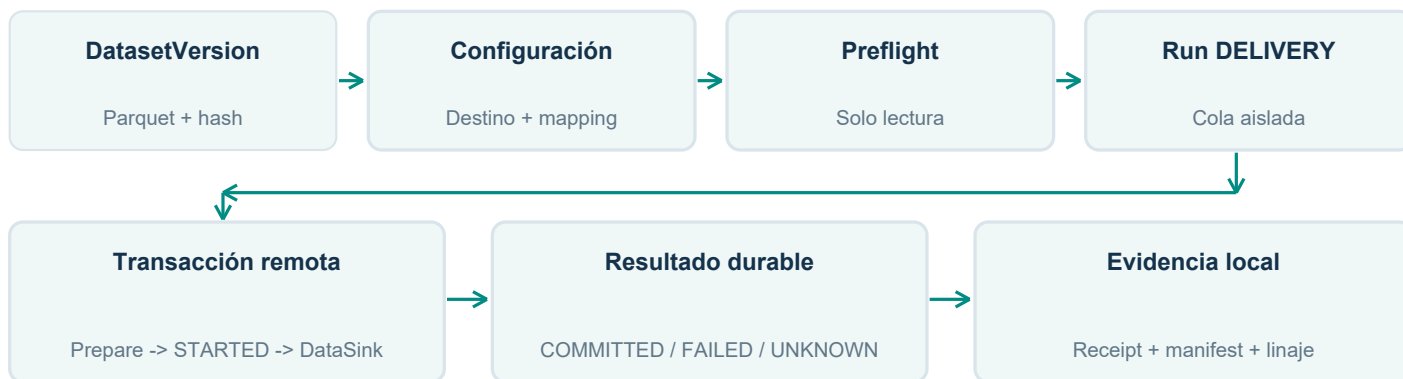
Un nuevo adaptador implementa DataSource y su normalización, registra opciones permitidas y aporta pruebas de permisos, límites, tipos y trazabilidad. Fuentes no SQL tendrán DTOs específicos, sin forzar el concepto de schema/tabla sobre buckets o APIs. El motor de calidad no cambia. Data Delivery implementa su puerto DataSink independiente; DataSource continúa sin incorporar write ni entrega hacia sistemas externos.

5B. Data Delivery

IMPLEMENTADO. Data Delivery publica una DatasetVersion canónica e inmutable en PostgreSQL o SQL Server. La versión 0.5.1 endurece la evidencia, la interpretación operativa de UNKNOWN y las métricas. No incorpora transformación funcional, conectores nuevos, automatización de entregas ni una transacción distribuida.

A. Propósito y flujo funcional

El usuario escoge una versión concreta, un destino versionado, el target y el mapping. La vista previa explica la proyección; el preflight inspecciona el destino sin escribir. Publicar fija una configuración inmutable. Ejecutar crea una Run y un Job DELIVERY; el worker repite las comprobaciones antes de abrir la transacción. El resultado durable precede a receipt y manifest.



La fuente no se consulta otra vez durante la entrega: el mismo snapshot conserva sus valores aunque una tabla fuente o un archivo original cambien. Una versión aceptada de Intake también puede ser input; su linaje sigue apuntando al Intake que la produjo. La entrega no significa que las reglas de negocio del receptor hayan aprobado el contenido.

B. Identidad y versiones del destino

DeliveryDestination mantiene identidad estable, nombre, motor, estado enabled, revisión optimista y último test.

DeliveryDestinationVersion conserva una revisión inmutable con configuración técnica y hash. Una configuración fija ambos IDs; cambiar la contraseña, host o nombre del destino no reescribe runs históricos.

La API recibe password pero no la devuelve ni expone secret_reference. La contraseña cifrada vive en delivery_credentials y la clave en delivery_keys, volúmenes diferentes de los secretos fuente. La referencia opaca participa en el hash interno, no aparece como credencial pública. Deshabilitar o eliminar lógicamente el destino impide nuevas entregas; no borra su historia.

C. Contrato DataSink y límites de responsabilidad

Operación	Responsabilidad
test / schemas / tables	Comprobar conectividad y descubrir objetos autorizados; no escribir negocio.
table_metadata / permissions	Obtener tipos, constraints, collation y capacidades necesarias.
prepare	Validar identificadores, convertir representación técnica del mismo tipo lógico y congelar PreparedDelivery localmente.
deliver_prepared	Ejecutar la estrategia y confirmar o clasificar el resultado de la transacción remota.
DeliveryResult	Métricas de la operación confirmada y referencia remota opcional; no inventario físico final.

DatasetSource sólo lee. StorageProvider sólo administra evidencia interna. DataSink no evalúa reglas Intake/Recon/Sentinel ni modifica el Parquet canónico. PreparedDelivery contiene schema, tabla, columnas, tuplas, estrategia, claves y bytes preparados. La preparación ocurre antes del marcador STARTED: un error local de tipos no debe convertirse en una supuesta transacción ambigua.

D. Mapping, selección y orden

El mapping declara source_name, target_name, target_type, ordinal y nullable; STRING añade length y DECIMAL admite precision/scale. Puede seleccionar, reordenar y renombrar. No puede inventar columnas, repetir nombres o interpretar STRING como número, fecha, timestamp o booleano.

Tipo lógico	Target nuevo PostgreSQL	Target nuevo SQL Server	Regla de conservación
STRING	text / varchar	nvarchar con collation suplementaria	Texto Unicode exacto; identificadores conservan ceros iniciales.
INT64	bigint	bigint	Entero dentro del rango; no redondeo.
DECIMAL	numeric(p,s)	decimal(p,s)	Valor exacto, escala y dígitos enteros suficientes.
DATE	date	date	Fecha ISO inequívoca; no zona horaria.
TIMESTAMP	timestamptz(6)	datetimeoffset(6)	Offset explícito, instante conservado, hasta microsegundos.
BOOLEAN	boolean	bit	Booleano lógico; no interpretación libre de texto.

En target existente se inspecciona el tipo real y su capacidad. DECIMAL no se escribe en real/float aproximado. Un MONEY fuera del rango remoto puede superar las comprobaciones locales y fallar en la transacción: rollback comprobado, no éxito parcial. Identificadores SQL se validan y citan por dialecto; no se admite SQL libre en nombres ni una plantilla de consulta escrita por el usuario.

E. Texto y temporales: ejemplos exactos

STRING acepta almacenamiento variable Unicode exacto: text/varchar PostgreSQL y nvarchar SQL Server. Se rechazan NUL, surrogates no emparejados, tipos fixed y familias no Unicode. La longitud se comprueba conservadoramente en unidades UTF-16; caracteres suplementarios exigen collation _SC/_UTF8 en SQL Server. Tablas nuevas usan Latin1_General_100_CI_AS_SC.

Fuente nativa	Lógico conservado	Ejemplo / consecuencia
PostgreSQL timestampz(0..6)	TIMESTAMP	2026-09-20T11:30:00.123456-05:00 conserva el instante; la representación del offset puede normalizarse.
PostgreSQL timestamp without time zone(0..6)	STRING	2026-09-20 11:30:00.123456 no adquiere UTC implícita.
SQL Server datetimeoffset(0..6)	TIMESTAMP	Zona explícita y fracción hasta seis cifras.
SQL Server datetimeoffset(7)	STRING	CONVERT estilo127 conserva el séptimo dígito y puede representar el instante normalizado a UTC, por ejemplo 2026-09-20T16:30:00.1234567Z.
SQL Server datetime2(0..7), datetime, smalldatetime	STRING	Sin zona; conservar texto nativo, no atribuir un instante.

La regresión 0.5.1 recorre adquisición, refresh, lectura histórica, Intake, Sentinel, Recon y Delivery en ambos motores. El objetivo no es cambiar esta política sino detectar pérdida de fracción, offset o coerción implícita. Si una versión conserva un temporal como STRING, Data Delivery sólo lo publica como STRING; convertirlo a TIMESTAMP requiere otra capacidad explícita, fuera de este endurecimiento.

F. Target nuevo o existente

EXISTING_TABLE exige schema y tabla existentes, columnas compatibles, permisos y constraints adecuados. CREATE_TABLE se combina exclusivamente con CREATE_AND_LOAD. create_schema=true significa crear un schema que todavía no existe; no autoriza adoptar uno aparecido concurrentemente.

El preflight no reserva el nombre. La transacción vuelve a comprobarlo: si otra sesión creó el schema o target entre validación y escritura, falla de forma controlada. Ni la carrera ni un reintento convierten CREATE_AND_LOAD en overwrite. No se ejecuta DROP TABLE para acomodar una estructura incompatible.

G. CREATE_AND_LOAD

Crea el target con la definición técnica validada y carga la población preparada en una transacción. Cuando se solicitó, incluye la creación del schema. Un error de DDL o una fila rechazada revierte el conjunto conforme a las garantías del motor; no se publica un receipt COMMITTED antes de confirmar commit.

Para cero filas puede crear una tabla vacía: filas preparadas y enviadas son cero. Esto no demuestra que otros volúmenes sean soportados. La metadata de la nueva tabla se gobierna por el mapping, no por una inferencia distinta dentro del sink.

Precondiciones y permisos: schema existente con USAGE/CREATE en PostgreSQL, o CREATE de base si se crea schema; SQL Server requiere CREATE TABLE y ALTER del schema existente, o CREATE SCHEMA si es nuevo. El target debe estar ausente. La operación usa los locks DDL del motor, no reserva previamente el nombre. TARGET_ALREADY_EXISTS o SCHEMA_NOT_FOUND abortan ante drift; fallos de permisos o constraints no se convierten en carga parcial confirmada.

Idempotencia y métricas: reabrir la misma Run no recrea la tabla ni reenvía datos. Otra Run deliberada contra el mismo nombre falla por existencia. Tras COMMITTED, rows_written=N; rows_inserted usa el conteo conocido del driver o null y rows_updated=0. Confirmación de commit perdida produce UNKNOWN, no un replay DDL.

H. APPEND

Inserta todas las filas preparadas en un target existente. No elimina ni compara el contenido anterior. Dos runs deliberadamente diferentes pueden anexas las mismas filas: la idempotencia de Trackvance evita duplicar una misma solicitud identificada, no convierte APPEND en deduplicación remota.

rows_written describe la población fuente enviada en la operación confirmada. Si había 100 filas y se enviaron 20, el contador no se convierte en 120. Un trigger puede además suprimir, transformar o generar filas; ese efecto no se deduce del contador.

Precondiciones y permisos: estructura compatible y columnas requeridas satisfechas; PostgreSQL exige INSERT, SQL Server INSERT+SELECT. PostgreSQL toma ROW EXCLUSIVE para estabilizar el target; SQL Server TABLOCKX/HOLDLOCK hasta fin de transacción. El lock SQL Server puede bloquear otros escritores/lectores y no implica un rendimiento ilimitado. Constraints, timeout o target ausente producen error saneado y rollback conocido cuando el motor lo confirma.

Resultado y métricas: no hay actualizaciones explícitas (`rows_updated=0`); inserciones sólo se reportan si el driver conoce su conteo. La población previa no se suma. La pérdida de confirmación conserva UNKNOWN y exige verificar el destino antes de otra Run, pues volver a anexar puede duplicar filas.

I. OVERWRITE

Ejecuta DELETE e inserción en una misma transacción; conserva la tabla y no hace drop/recreate. Sustituye toda la población autorizada del target. Si falla la carga, un rollback conocido debe conservar la población anterior.

Se rechazan RLS activa en PostgreSQL y FILTER security policies en SQL Server, porque podrían dejar filas no visibles al usuario y falsear la idea de reemplazo. Permisos INSERT y DELETE son necesarios; SQL Server requiere también SELECT y VIEW DEFINITION para las verificaciones. Un input vacío borra la población existente dentro de la transacción: debe ser una elección deliberada.

Locks: SHARE ROW EXCLUSIVE en PostgreSQL; TABLOCKX/HOLDLOCK en SQL Server. La validación de políticas se repite bajo lock. UNSAFE_ROW_SECURITY o falta de visibilidad completa impiden la operación; no se considera válido borrar sólo las filas que una policy deja ver. IGNORE_DUP_KEY en SQL Server también se rechaza. Puede haber contención, crecimiento de log y fallos de constraints; la estrategia no deshabilita triggers ni integridad referencial.

Métricas e idempotencia: `rows_written` cuenta N filas enviadas, no filas borradas; `rows_inserted` depende del driver y `rows_updated=0`. Otra Run es otra sustitución, no una continuación automática. Si no se conoce el resultado del commit, la revisión externa precede a cualquier nueva sustitución deliberada.

J. UPSERT y claves compuestas

Exige claves explícitas, incluidas en el mapping, sin null ni duplicados en la fuente. El destino debe respaldarlas con PK o UNIQUE compatible. Una clave compuesta es una tupla ordenada con comparación nativa; no se concatena texto con un separador que pueda provocar colisiones.

El staging de claves usa tipos y collation del target para descubrir colisiones que la igualdad de Python no detectaría. No se acepta una restricción distinta porque parezca equivalente después de un cambio concurrente. Un input vacío sigue validando y bloqueando el target; cero filas no desactiva la seguridad.

Permisos: INSERT+UPDATE+SELECT en ambos motores, más TEMPORARY de base en PostgreSQL y acceso a #temp conforme a tempdb en SQL Server. PostgreSQL toma ROW EXCLUSIVE, revalida el árbitro nombrado y ejecuta ON CONFLICT; SQL Server conserva TABLOCKX/HOLDLOCK y UPDLOCK/HOLDLOCK para la búsqueda/actualización, sin MERGE.

TARGET_KEY_NOT_UNIQUE y drift de constraint impiden confirmar una coincidencia ambigua; los errores de constraints y duplicados revierten el lote.

Una repetición como Run nueva puede modificar otra vez valores o disparar triggers: UPSERT no garantiza ausencia universal de efectos secundarios. `rows_written=N`; insertadas/actualizadas siguen las capacidades descritas en K/L, no se infieren de una consulta previa. Vacío informa cero acciones conocidas.

K. PostgreSQL: transacción y conteos fiables

El adaptador conserva el lock del target y revalida nombre/columnas de la constraint después de adquirirlo. UPSERT exige una PK/UNIQUE nombrada no diferible y usa ON CONFLICT ON CONSTRAINT; no sustituye la operación atómica por un SELECT seguido de UPDATE/INSERT susceptible a carreras.

En PostgreSQL 18 la cláusula RETURNING documentada permite observar OLD/NEW. La detección de inserción usa la ausencia del registro OLD completo, no sólo una clave nullable; un registro existente cuyos campos sean null sigue siendo un registro existente. Los conteos son acciones DML de nivel superior reportadas por el motor. BEFORE triggers que suprimen filas no se cuentan como inserciones físicas exitosas; los efectos secundarios de triggers no son un censo final.

En PostgreSQL 16/17 no hay una descomposición fiable equivalente para UPSERT no vacío con columnas actualizables: `rows_inserted` y `rows_updated` quedan null/N/D. La excepción es el mapping formado sólo por claves, que ejecuta DO NOTHING ante conflictos: las inserciones usan el affected-row count conocido y las actualizaciones son cero. Una operación vacía también puede afirmar cero porque no hubo acciones. No se usa xmax, estadísticas aproximadas ni un SELECT previo para fabricar precisión.

Referencias primarias: <https://www.postgresql.org/docs/18/dml-returning.html> y <https://www.postgresql.org/docs/18/sql-insert.html>. La base metadata y el destino baseline siguen en PostgreSQL 16; el runner añade un destino desechable 18, sin actualizar silenciosamente la instalación del usuario.

L. SQL Server: concurrencia y conteos

El adaptador evita MERGE. Usa transacción, staging #temp con tipos/collation del target, locks y comprobación por clave; SELECT @@ROWCOUNT valida que una acción no afecte más de una fila. La disponibilidad de tablas temporales depende de la política de tempdb. Un índice unique con IGNORE_DUP_KEY se rechaza para no silenciar duplicados.

Los conteos sólo se exponen cuando el adaptador puede sostenerlos. Si el driver no proporciona un affected-row count conocido, se utiliza null, no el tamaño de la fuente ni un cero ficticio. Los efectos secundarios de triggers no se deducen de esos contadores. La semántica de rows_written continúa siendo filas fuente enviadas en la operación confirmada, no la suma de todas las filas tocadas por triggers.

M. Familias de preflight y fallos

Familia	Comprobación / riesgo prevenido
Fuente e integridad	DatasetVersion propia, artifact canónico materializable, tamaño y SHA-256; no entregar bytes sustituidos.
Destino	Revisión exacta, hash, enabled y conexión; no usar otro endpoint por una edición posterior.
Mapping y valores	Columnas, preservación de tipo, identificadores, nullability, longitudes, precisión y escala.
Target	Existencia/ausencia esperada, metadata, columnas generadas/identity/default y compatibilidad real.
Permisos	Privilegios específicos de estrategia, schema, tabla y temporales.
UPSERT	PK/UNIQUE compatible; claves fuente no nulas, no duplicadas y collation nativa.
Recursos locales	Consistencia del número real de filas/columnas con la metadata, límite MAX_ROWS y hasta 100 columnas. No comprueba memoria ni espacio libre del host.

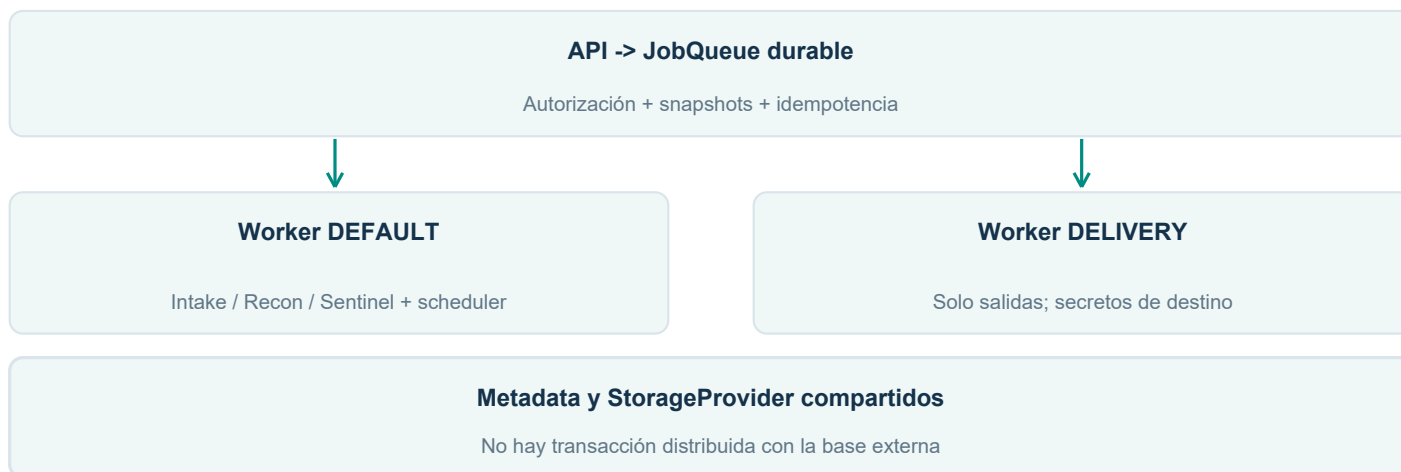
POST /preflight realiza lectura técnica; el resultado no garantiza que nada cambie después. El worker lo repite, y los guards transaccionales son la última autoridad ante drift. FAILED_PRECONDITION es un estado del Run anterior al intento remoto: no debe confundirse con DeliveryAttempt.FAILED después de STARTED.

N. Preview, configuración y publicación

POST /preview muestra una muestra acotada antes/después del mapping, orden y nombres. No envía datos al destino ni crea un intento. No certifica que todas las filas satisfagan constraints externas. La publicación de configuración valida el draft y fija schema_version=1, DatasetVersion, DestinationVersion, target, columns, write_strategy y upsert_keys.

Una nueva revisión es otra configuración inmutable. La UI no puede cambiar target_type para forzar una conversión lógica prohibida. El propietario visible es una etiqueta operativa y no sustituye permisos o identidad de usuario. config_hash y stored_config_hash permiten verificar lo publicado.

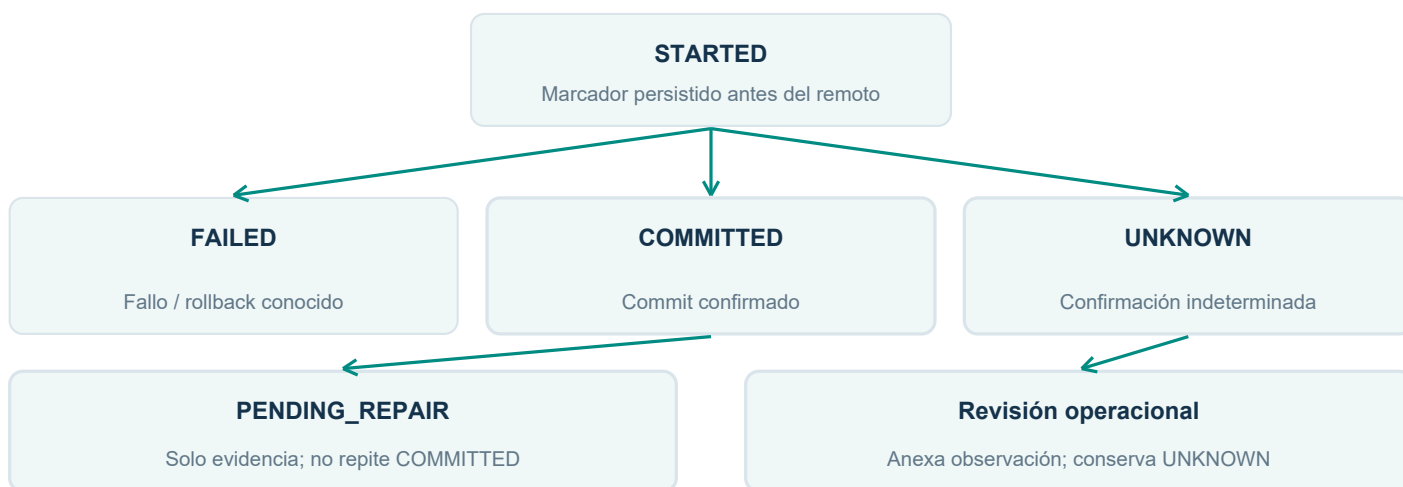
O. Workers, lanes y frontera de commit



DEFAULT procesa Intake/Recon/Sentinel y programación local. DELIVERY sólo procesa entregas y no ejecuta scheduler ni monta secretos fuente. Ambos comparten metadata y StorageProvider; el delivery-worker sólo monta secretos de destino. El Job incluye lane y una reclamación condicionada al lease.

La adquisición/reconciliación conserva el orden Run -> Job. Antes de STARTED se comprueba de nuevo que el worker conserva la reclamación. Una cancelación previa impide iniciar la escritura; un COMMITTED o FAILED ya conocido prevalece sobre una cancelación tardía. PostgreSQL interno y el motor receptor no participan en 2PC: perder la confirmación exige expresar incertidumbre, no repetir.

P. Estados del intento y del Run



Hecho verificable	Attempt	Run / decisión
Preflight o preparación local fallidos	No se inicia intento	FAILED_PRECONDITION / FAILED
Marcador durable previo al remoto	STARTED	RUNNING
Commit remoto confirmado y persistido	COMMITTED	SUCCESS / COMMITTED
Rechazo o rollback conocido	FAILED	FAILED / FAILED
Confirmación perdida o STARTED recuperado sin prueba	UNKNOWN	UNKNOWN / UNKNOWN
Commit conocido; publicación local incompleta	COMMITTED	SUCCESS / COMMITTED + PENDING_REPAIR

UNKNOWN no es una forma de FAILED. El sistema no afirma rollback ni commit cuando no puede demostrarlos. La recuperación de un attempt ya UNKNOWN no completa ni corrige retrospectivamente sus campos históricos. La revisión operacional es un registro aparte.

Q. UNKNOWN: revisión operacional append-only

GET y POST /api/v1/delivery/runs/{run_id}/reviews consultan/anexan observaciones. El cuerpo exige delivery_attempt_id, outcome y note; verified_at es opcional, con zona, no futuro ni anterior al inicio del intento. Si se omite, el servidor registra el instante actual. La nota debe contener texto y tiene un máximo de 4.000 caracteres.

outcome	Interpretación, nunca nuevo estado del intento
REMOTE_COMMIT_OBSERVED	El operador aporta una observación externa compatible con commit.
REMOTE_NOT_COMMITTED_OBSERVED	El operador aporta una observación externa de ausencia de commit.
INCONCLUSIVE	La comprobación no permite decidir.

DeliveryOperationalReview conserva org, Run, attempt, reviewer_id, snapshot de nombre, outcome, nota, verified_at y created_at. La migración nueva 0009_delivery_reviews crea sólo esta tabla con FKs, índices y CHECK de outcomes. No modifica 0001..0008. No hay PATCH/DELETE de revisiones: una corrección se anexa como nueva observación. No existe aprobación de cuatro ojos ni validación remota automática de lo escrito por el operador.

La UI muestra historial y advertencia de incertidumbre. Guardar una revisión no cambia Run/attempt a COMMITTED o FAILED, no encola un Job, no crea otro Run y no llama al DataSink. Si luego procede una nueva entrega, debe crearse deliberadamente con una nueva decisión operativa; no hay botón de replay ciego.

R. COMMITTED + PENDING_REPAIR: sólo evidencia local

POST /api/v1/delivery/runs/{run_id}/repair-evidence reconstruye receipt/manifest cuando existe Run DELIVERY SUCCESS/COMMITTED y un único attempt COMMITTED coherente. No se admite reparar UNKNOWN, FAILED, otro módulo o una identidad ajena. El resultado es REPAIRED o ALREADY_VALID con los IDs de ambos artifacts.

La operación bloquea la Run, valida organización, configuración/hash, identidad del destino/version, source schema/hash, artifact canónico y métricas persistidas. Usa los datos ya confirmados; no solicita contraseña, no descifra secretos, no instancia DataSink, no abre conexión remota y no repite DDL/DML/commit.

Los IDs nuevos de evidencia se derivan de org/Run/kind. Se reutilizan bytes y enlaces válidos; las restricciones de linaje y la publicación sin reemplazo evitan duplicación. Ante publicación parcial puede completar lo ausente. Un archivo registrado ausente sólo se reconstruye si los bytes coinciden con su SHA histórico; un archivo corrupto o una discrepancia no se sobrescribe para ocultar el problema. El servicio falla cerrado y registra el fallo.

Los planes 0.5.1 fijan además source_identity y la versión del motor de evidencia. Planes 0.5.0 que no contienen ese bloque se verifican con los registros retenidos; no se inventa un hash independiente que nunca fue almacenado. Artefactos 0.5.0 válidos se conservan byte por byte. La UI ofrece “Reparar evidencia”, nunca “Reintentar entrega”, y mantiene visible el commit confirmado.

Existe una ventana normal entre el commit durable y la publicación local. Si SUCCESS/COMMITTED todavía no incluye receipt_artifact_id ni PENDING_REPAIR, la UI muestra “Publicando evidencia local”, deshabilita descargas y consulta el Run cada 1,5 segundos durante hasta 60 segundos. Una respuesta lenta no se cancela para iniciar otra. Al vencer ese plazo ofrece Actualizar estado, sólo lectura; no marca la entrega como fallida ni ejecuta reparación o replay automáticamente. Si llega PENDING_REPAIR, pasa a la acción explícita de reparación con su permiso.

S. Idempotencia y concurrencia

Idempotency-Key identifica una solicitud de ejecución con el mismo payload y scope; repetirla devuelve la misma Run, no otra transacción. No es una clave de deduplicación de negocio remota ni autoriza repetir UNKNOWN. Configuración, DatasetVersion y destino versionado forman el contexto verificable.

La reparación es idempotente respecto a artifacts, enlaces y resultado; cada petición puede añadir auditoría de la acción, que no debe confundirse con otra entrega. La serialización local, los IDs estables y los guards de publicación protegen incluso una interrupción entre bytes y metadata. La revisión UNKNOWN, en cambio, es un historial de observaciones: cada POST válido añade un registro.

T. Métricas: significado y límites

Campo	Significado 0.5.1
rows_attempted	Filas fuente preparadas para el intento.
rows_written	Filas fuente enviadas sin error en una operación cuyo commit fue confirmado. No es COUNT(*) remoto.
rows_inserted	Acciones INSERT fiables reportadas por el adaptador; null cuando no puede garantizarlas.
rows_updated	Acciones UPDATE fiables reportadas por el adaptador; null cuando no puede garantizarlas.
bytes_sent	Tamaño UTF-8 preparado de valores no null; no bytes de red/TLS ni tamaño final de la base.
preflight_seconds	Duración monótonica del preflight del worker en runs nuevos.
write_seconds	Duración monótonica de deliver_prepared, incluida confirmación remota, excluida publicación local.

null se muestra como N/D. Cero es un conteo conocido, nunca una sustitución cosmética de null. Los nuevos DTOs/evidencias añaden `metric_semantics` versión 1; la lectura de manifests 0.5.0 sin el bloque sigue siendo válida y no reescribe bytes. Los timings históricos ausentes no se rellenan con cero.

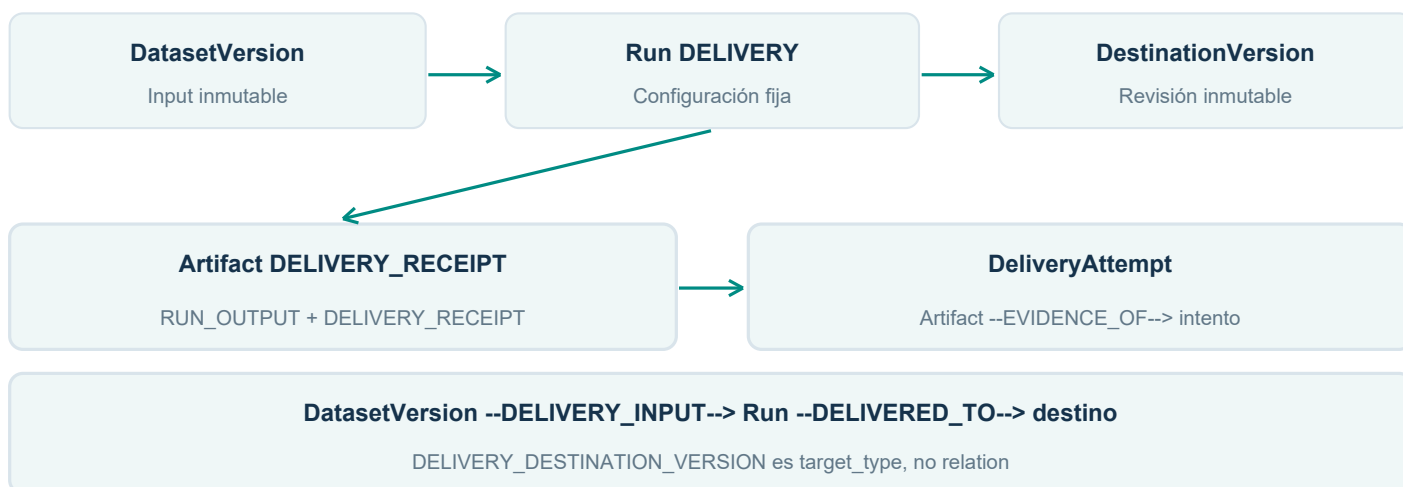
U. Receipt y manifest: ejemplo abreviado

Ejemplo de forma, no resultado de una certificación. IDs y hashes son marcadores; los conteos elegidos ilustran UPSERT PostgreSQL 16 con desglose no disponible.

```
{
  "schema_version": 1,
  "kind": "DELIVERY_RECEIPT",
  "run_id": "<run>",
  "dataset_version_id": "<snapshot>",
  "canonical_artifact_id": "<artifact>",
  "canonical_sha256": "<sha256>",
  "destination_id": "<destino>",
  "destination_version_id": "<revision>",
  "sink_type": "POSTGRESQL",
  "target": {
    "mode": "EXISTING_TABLE",
    "schema_name": "public",
    "table_name": "records",
    "create_schema": false
  },
  "write_strategy": "UPSERT",
  "delivery_attempt_id": "<attempt>",
  "attempt_number": 1,
  "rows_attempted": 3,
  "rows_written": 3,
  "rows_inserted": null,
  "rows_updated": null,
  "result": "COMMITTED"
}
```

El receipt completo añade fuente, destino/config_hash, nombres/versiones, fechas, bytes y `metric_semantics`. El manifest schema 2 incluye inputs, configuration, processing, actor, engine_version, metrics y `result_artifacts`. Su sección `delivery` incluye `destination`, `target`, `write_strategy`, metrics y `attempt`. Repara con los valores persistidos del commit, no con un nuevo timestamp de ejecución.

V. Linaje persistido y auditoría



source_type -> target_type	relation real
DATASET_VERSION -> RUN	DELIVERY_INPUT
RUN -> DELIVERY_DESTINATION_VERSION	DELIVERED_TO
RUN -> ARTIFACT (receipt)	DELIVERY_RECEIPT
ARTIFACT (receipt) -> DELIVERY_ATTEMPT	EVIDENCE_OF
RUN -> ARTIFACT (receipt o manifest)	RUN_OUTPUT

DELIVERY_DESTINATION_VERSION es un tipo de entidad, no la relación. Esta corrección documental no migra enlaces históricos ni cambia sus direcciones. Los tests contrastan los triples persistidos y la reconstrucción de evidencia. El backfill legacy de arranque no añade RUN_INPUT genérico a Runs Delivery; preserva los vínculos históricos que ya existan y no reemplaza la reparación explícita. El restore compara el grafo completo, sin omitir enlaces para aprobar.

Auditoría conserva DESTINATION_CREATED/TESTED, DELIVERY_CONFIGURATION_PUBLISHED, DELIVERY_RUN_QUEUED, DELIVERY_STARTED/COMMITTED/FAILED/UNKNOWN y, en 0.5.1, DELIVERY_EVIDENCE_REPAIR_STARTED, DELIVERY_EVIDENCE_REPAIRED, DELIVERY_EVIDENCE_REPAIR_FAILED y DELIVERY_UNKNOWN_REVIEWED. No se incluye la nota libre de revisión en metadata de audit, aunque permanece accesible en su recurso autorizado.

W. Permisos, secretos y aislamiento

Se reutilizan connections:read/manage/use, configurations:write, runs:read/execute y artifacts:download. Reparar y registrar revisión exigen runs:execute; leer revisiones exige runs:read. Las mutaciones conservan CSRF, sesión y scope de organización en backend. No se añaden roles ni permisos nuevos.

No se permite cruzar un attempt de otra Run u organización. El nombre visible del revisor es snapshot y su UUID estable se conserva. UI oculta acciones no autorizadas, pero el backend sigue siendo la autoridad. RBAC granular por destino/estrategia y aprobación de cuatro ojos quedan fuera de alcance.

X. Backup, restauración y compatibilidad

El backup conserva PostgreSQL, artifacts, secretos fuente/destino y ambas claves. Manifest de backup sigue en versión 2 porque la topología no cambia; la huella de estado pasa a 4 para incluir delivery_reviews. El restore nativo exige igualdad exacta de registros, hashes, secretos y relaciones.

Los backups 0.5.0 (state 3, 0008) se migran a 0009 y se comparan mediante snapshot-legacy-v3, que excluye únicamente la nueva tabla vacía. Los 0.4.x (state 2, 0007) conservan la proyección legacy-v2, excluyendo tablas Delivery vacías y el lane DEFAULT añadido. No se omiten filas de revisiones para fingir compatibilidad: si existen en un upgrade legacy, la normalización falla.

El drill 0.5.1 preservó nueve artifacts, dos secretos, 164 relaciones, dos intentos y una revisión. Antes del backup, dos peticiones de reparación concurrentes devolvieron REPAIRED/ALREADY_VALID con los mismos IDs y bytes. Después del restore, otra reparación devolvió ALREADY_VALID; el target original conservó cuatro filas y no se repitió la escritura. Se destruyó el origen desechable antes de restaurar en un proyecto nuevo. La revisión UNKNOWN fue un fixture explícito SIMULATED_UNKNOWN_NO_REMOTE_IO: no certifica una pérdida real exacta de confirmación post-commit; ese escenario se reporta separadamente, sin convertir NOT_RUN en PASS.

Y. Benchmark dedicado y operación en UI

scripts/tests/delivery_benchmark_cycle.py levanta destinos PostgreSQL y SQL Server reales y desechables, usa contenido SHAKE256 determinista variado y recorre CREATE_AND_LOAD, APPEND, OVERWRITE y UPSERT. Smoke: 1 MiB y 1.000 filas. Representativo acotado: 8 MiB y 20.000 filas. Ninguno certifica capacidad de producción. Los límites habituales de 10 MiB/100.000 filas no aumentan.

Registra bytes/filas/columnas/hash de origen; tiempo total, preflight y escritura; filas/s y MB/s; CPU, memoria, I/O y crecimiento de storage/temporales; outcome, conteos y motivo real de parada. Los picos muestreados pueden omitir ráfagas breves y los contadores cgroup incluyen actividad incidental del contenedor. Recurso insuficiente produce NOT_RUN_RESOURCE_LIMIT, nunca PASS. Los resultados medidos se publican en el informe específico y JSON sanitizados 0.5.1.

Resultados locales del 25 de septiembre de 2026

El smoke real de 1.074.923 bytes / 1.000 filas aprobó las ocho combinaciones en 143,59 s de wall time. El representativo de 8.917.809 bytes / 20.000 filas también aprobó ocho de ocho en 207,70 s. Ambos tienen cuatro columnas y payload distinto por fila; el representativo usa celdas de 420 caracteres, frente a 1.049 en smoke. No son repeticiones estadísticas ni una progresión a ancho de fila constante.

Motor / estrategia	Escritura smoke s	Escritura 20.000 filas s	Filas/s representativo
PostgreSQL 16 / CREATE_AND_LOAD	0,0255	0,2082	96.071
PostgreSQL 16 / APPEND	0,0168	0,1945	102.807
PostgreSQL 16 / OVERWRITE	0,0247	0,2191	91.273
PostgreSQL 16 / UPSERT	0,0392	0,6329	31.599
SQL Server 2022 / CREATE_AND_LOAD	0,4034	7,1269	2.806
SQL Server 2022 / APPEND	0,4097	7,1833	2.784
SQL Server 2022 / OVERWRITE	0,4727	7,2688	2.751
SQL Server 2022 / UPSERT	1,7187	31,7338	630

Escritura incluye conexión, locks/checks, DML y commit dentro de deliver_prepared; no incluye el preflight anterior, cola ni publicación local. El total de caso sí incluye esos pasos y polling, pero excluye arranque, upload y SQL independiente de verificación. El informe dedicado contiene ambos preflights, total, MB/s, CPU/I/O por contenedor y método, sin mezclar estas ventanas.

En la carga representativa se prepararon/enviaron 20.000 filas por caso; UPSERT SQL Server reportó 10.000 insertadas y 10.000 actualizadas. PostgreSQL 16 devolvió null/null, aunque el SQL independiente verificó la mezcla esperada. APPEND dejó 40.000 filas físicas; las demás estrategias dejaron 20.000. Esto confirma por qué filas enviadas no significa tamaño final de la tabla.

El pico de memoria agregado muestreado fue 1.251.307.681 bytes en smoke y 2.050.125.461 en representativo; CPU agregada máxima 141,38 % y 213,85 % de un núcleo, respectivamente, sobre 16 CPU lógicas asignadas a Docker. Hubo 28 y 40 muestras, sin OOM ni corte del watchdog. El delivery-worker alcanzó 217.186.304 bytes en el representativo; storage de artifacts creció de 15.325.905 a 15.407.176 bytes. El temporal observado fue cero, sin garantizar ausencia de picos entre muestras. El cgroup memory.peak es acumulado, no exclusivo del caso.

Los tiers de 100/500 MiB y 1/2/5 GiB quedan NOT_RUN_RESOURCE_LIMIT por el límite vigente de entrada de 10 MiB, no por una medición de incapacidad del hardware. El primer smoke falló antes de escribir porque el mapping del fixture usaba INT64 donde la inferencia produjo DECIMAL; se conserva como FAIL y su repetición es un resultado separado. La corrección no alteró tipos ni límites del producto.

Evidencia: docs/development/delivery-benchmark-results-0.5.1.md y los directorios delivery-benchmark-smoke, delivery-benchmark-smoke-retry y delivery-benchmark-representative bajo docs/development/evidence/0.5.1.

Recorrido funcional de la interfaz

Pasos de UI: crear/probar destino; escoger versión; configurar target/mapping; revisar preview/preflight; publicar; ejecutar y abrir detalle. El detalle muestra intento, filas enviadas/N/D, receipt y manifest. Ante PENDING_REPAIR, comprobar el commit y pulsar Reparar evidencia. Ante UNKNOWN, verificar externamente, elegir outcome, explicar la observación y guardar; revisar el historial sin ejecutar una entrega adicional implícita.

Z. Límites y criterio de aceptación

No hay garantía exactly-once distribuida, replay automático de UNKNOWN, SQL arbitrario, transformación funcional, scheduling Delivery, nuevo conector, SHIST, correo, SSO, HA ni certificación de volumen productivo. Los efectos secundarios del receptor quedan fuera del conteo físico de Trackvance.

La aceptación exige tests nuevos de reparación sin DataSink, historial UNKNOWN inmutable, semántica nullable, conteos reales por versión de motor, temporalidad sin coerción, linaje exacto, benchmark y code splitting. Además se repiten las suites base,

migraciones, conexiones, Delivery, recovery, navegador y CI. El apartado de validación contiene resultados ejecutados; este diseño no sustituye esa evidencia.

6. Carga, esquema y versionado

El flujo previo a guardar una versión detecta formato, hoja/delimitador, columnas y tipos. El usuario puede corregir el tipo lógico o marcar identificadores. Los mismos controles se ofrecen al crear una versión de un dataset existente.

- Columnas identificadoras se selecciona únicamente desde el esquema inspeccionado, con opción Todos. Se eliminó la entrada redundante de nombres libres; cada selección conserva el override `STRING + IDENTIFIER`.
- Los tipos disponibles son `STRING`, `INT64`, `DECIMAL`, `DATE`, `TIMESTAMP` y `BOOLEAN`; un override explícito se valida contra los valores del archivo.
- El campo Área de negocio permite agregar un área nueva. El listado muestra origen entre Dataset y Área: Manual para carga, Data Intake para salida derivada.
- El listado combina búsqueda/filtro y orden ascendente/descendente por Área, Registros, Versiones, Estado y Última actualización.
- Un nombre ya usado en la organización devuelve `DATASET_NAME_EXISTS` y permite orientar al usuario hacia una versión nueva.

Secuencia persistente

1. La API escribe el upload en staging limitado del provider y detecta el lector.
2. El lector produce el modelo común; profiling valida inferencia/overrides.
3. `StorageProvider` conserva `ORIGINAL_UPLOAD` con hash, tamaño y MIME.
4. Se publica `CANONICAL_PARQUET` y se crea `DatasetVersion` inmutable con esquema, perfil e `ingestion_metadata`.
5. `ArtifactLink` registra relaciones y `AuditEvent` registra `DATASET_UPLOADED`. Los temporales del flujo normal se eliminan.

`DatasetVersion` conserva `dataset_id`, `version`, `filename`, `source_type`, `SHA-256`, `schema_hash`, filas/columnas, `profile_status`, `schema_json`, `profile`, `ingestion_metadata`, `parent_version_id`, `original_artifact_id`, `canonical_artifact_id` y `source_run_id`. `original_path/canonical_path` se conservan como metadata interna compatible, no se exponen como rutas del cliente.

`ingestion_metadata` incluye `reader.key/version`, `source_format`, `format_label`, `reader_options` efectivas, esquema nativo y política de numeración. Los históricos reciben un objeto vacío y siguen el adaptador compatible sin recalculando sus perfiles.

`GET /datasets/{id}/schema` usa por defecto el perfil persistente. `refresh=true` vuelve a leer el footer canónico y conserva tipos lógicos declarados; devuelve `scan_mode/scanned_rows` y no aplica transforms ni reescribe historia. El control `SHA-256` puede leer bytes para integridad aunque no procese filas de negocio.

Caso operativo: snapshot, refresh e historial

Una carga crea `DatasetVersion v1` y registra original y canónico. Un refresh de fuente crea `v2` con otro ID y `REFRESH_OF` hacia `v1`, aunque el nombre del dataset sea el mismo. Abrir un Run antiguo sigue leyendo `v1` y sus hashes, no la tabla externa actual. Si la fuente deja de estar disponible, su historial materializado continúa legible, siempre que los artifacts conservados superen la integridad.

Corregir valores no modifica el archivo anterior: cargar/refreshar una nueva versión y ejecutar otra configuración/Run permite comparar el antes y después. Una pérdida o corrupción canónica no se corrige recalculando el `SHA` esperado; debe recuperarse desde un respaldo verificable o declararse no disponible.

7. Profiling, identificadores y transforms

La política actual es EXACT_OBSERVED, metric_definition_version=2. No se recortan espacios, cambia case ni normaliza Unicode para contar valores distintos. "Cliente 12" y " Cliente 12 " son valores distintos; las diferencias de mayúsculas y de representación Unicode se conservan. Null y texto vacío tienen semánticas propias del lector y del perfil; no se equiparan mediante un trim implícito.

Inferencia inequívoca

DATE se infiere cuando los valores no nulos válidos cumplen YYYY-MM-DD y representan fechas reales. Una fecha inválida o una mezcla incompatible impide inferir DATE. TIMESTAMP exige su semántica ISO diferenciada, con zona explícita según las reglas portables; no se adivinan fechas ambiguas como 01/02/2026.

IDENTIFIER tiene prioridad sobre la interpretación como magnitud. Los nombres id y *_id activan una heurística documentada; el usuario puede confirmar/override el esquema. "001234567" conserva su cero inicial frente a "1234567". No se convierten universalmente todas las columnas numéricas a identificadores. El esquema nativo Parquet ayuda a inferir tipos, incluso con columnas nulas, pero no invalida el override explícito.

En la carga inicial y al crear una versión, el selector de identificadores muestra las columnas realmente inspeccionadas y permite elegir algunas o todas. La API continúa recibiendo los mismos overrides; no se crea una segunda fuente de nombres ni se reescriben los esquemas de versiones históricas.

Los perfiles incluyen conteos de nulos y distintos, tasas y razón de unicidad, además de método/versión. Las series históricas LEGACY_NORMALIZED/1 se mantienen separadas de EXACT_OBSERVED/2 para no comparar métricas semánticamente incompatibles.

Transformaciones funcionales

Intake y ReconOps admiten listas ordenadas de transforms {column,type,parameters}: trim, case, unicode_normalization, empty_to_null, id_padding, remove_characters, decimal_parse y date_parse. Recon mantiene listas independientes para origen y destino. Sólo se ejecutan cuando el contrato las declara. El upload original, Parquet de entrada y perfil histórico no se reescriben; los resultados conservan los parámetros efectivos.

La interfaz presenta esos mismos tipos con nombres funcionales: Eliminar espacios externos, Convertir mayúsculas/minúsculas, Normalizar texto Unicode, Convertir texto vacío en nulo, Completar identificador, Eliminar caracteres, Interpretar número decimal e Interpretar fecha. Cada elemento muestra una explicación y un ejemplo recalculado con sus parámetros. Un preview de hasta ocho valores reales aplica las transformaciones de la columna en el mismo orden del contrato y muestra Antes / Después. Los controles permiten hacer explícito ese orden. Un parseo decimal o de fecha inválido muestra "No se pudo interpretar; la transformación conservó el valor que recibió". Los pasos posteriores siguen ejecutándose en el orden declarado. La vista previa de fechas cubre únicamente las directivas determinísticas que el navegador reproduce con certeza. Para formatos Python no soportados (por ejemplo %b, %j o %Z) muestra "Vista previa no disponible para este formato; la ejecución usará el formato declarado", sin convertir esa limitación en un fallo. datetime.strptime del backend conserva la autoridad sobre la ejecución. La vista previa se calcula sólo para informar: no crea artifacts, no modifica el DatasetVersion y no forma parte del resultado ni del hash de configuración.

Las configuraciones nuevas se validan y normalizan a schema_version=2. configuration_hash usa JSON determinista con claves ordenadas y SHA-256; stored_config_hash conserva además la identidad de la configuración almacenada. La adaptación legacy se centraliza en config_semantics y manifests, evitando condiciones de versión dispersas en módulos.

8. Reglas portables y Data Intake

RuleDefinition declara type, column o columnas en parameters, severity ERROR/WARNING, enabled, parameters, when opcional, code/message opcionales y rule_id estable. El código ausente se deriva de type. Publicar una regla nueva asigna rule_id; la lectura de configuraciones históricas nunca fabrica IDs ni altera sus fingerprints. La declaración se compila a una representación portable antes de evaluarse con el engine. No se admite scripting arbitrario ni Python eval/exec.

Tipo / código	Parámetros y semántica
required / REQUIRED	Campo presente, no null y no vacío. Whitespace no es vacío sin trim declarado.
not_null / NOT_NULL	Rechaza null.
unique / UNIQUE	Incumplen todos los registros duplicados; null_policy explícita.
compound_unique / COMPOUND_UNIQUE	Tupla de columns; todos los miembros de una combinación repetida incumplen, sin concatenar valores con separadores ambiguos.
numeric / NUMERIC	Decimal finito con punto; conserva precisión.
positive / POSITIVE	Decimal estrictamente mayor que cero.
type / TYPE	logical_type STRING, INT64, DECIMAL, DATE, TIMESTAMP o BOOLEAN.
range / RANGE	gt/gte y lt/lte, límites Decimal inequívocos.
length / LENGTH	min/max enteros inclusivos en puntos de código Unicode observados, sin trim ni conteo por bytes.
allowed_values / ALLOWED_VALUES	values con hasta 1000 escalares; pertenencia exacta.
regex / REGEX	Subconjunto portable Rust/RE2, flags i/m/s, máximo 500 caracteres.
date_rule / DATE_RULE	not_future, min/max ISO, timezone UTC y null_policy.
column_compare / COLUMN_COMPARE	Columna comparada contra other_column mediante eq/ne/gt/gte/lt/lte y logical_type STRING/DECIMAL/DATE/TIMESTAMP declarado.
reference / REFERENCE	Una o varias columns deben existir en reference_columns de una DatasetVersion explícita e inmutable.

null_policy admite ALLOW, FAIL o IGNORE según la regla. IGNORE excluye la fila de evaluated_count; no se cuenta como una evaluación aprobada. Los shorthand históricos numeric/positive conservan FAIL. El instante de referencia de fecha se fija al inicio del Run para que una misma ejecución no cambie de criterio entre registros.

Condiciones y referencias seguras

when acepta una condición tipada o un árbol all/any: hasta cuatro niveles, 64 nodos y diez condiciones por grupo. Los operadores son eq/ne/gt/gte/lt/lte/is_null/not_null. No hay código de usuario. Los literales se validan al publicar; una fecha o número mal formado no convierte silenciosamente una condición en falsa.

La condición filtra la población antes de evaluar la regla. En unicidad condicional se comparan únicamente las filas elegibles. Las excluidas por condición o IGNORE quedan en skipped_count. Ejemplo: country eq CO -> required department. El editor permite construir una condición simple con columnas reales; la API admite árboles acotados. El editor conserva una condición compuesta existente sin reinterpretarla.

REFERENCE fija dataset_version_id y correspondencia de columnas en el contrato. La API valida organización, existencia del snapshot y columnas; el worker materializa exclusivamente el Parquet verificado mediante StorageProvider. La comparación de tuplas conserva texto/null y no conecta desde el motor a una BD externa. El plan incluye el tamaño de las referencias; el manifest registra references y ArtifactLink registra RUN_REFERENCE. Un dataset de referencia vacío rechaza filas aplicables; null se gobierna por la política declarada.

Editor y resultados

Al elegir dataset, el editor carga su esquema. Columnas obligatorias, sin duplicados, numéricas y positivas usan multiselect con Todos; las positivas sólo permiten columnas numéricas. Las reglas avanzadas seleccionan columnas reales y muestran parámetros por tipo. Las referencias permiten elegir dataset, versión y columnas con su esquema persistido. El sistema detecta candidatos, pero el usuario decide las reglas. Actualizar esquema no obliga a escanear datasets completos. Responsable funciona como un catálogo deduplicado de owners de datasets y configuraciones del módulo; el usuario puede seleccionar uno o agregar una etiqueta nueva de hasta 120 caracteres. El contrato sigue persistiendo el mismo string owner: no es una cuenta, rol ni asignación de Excepciones.

Intake devuelve total_rows, valid_rows, error_rows, warning_rows, acceptance_rate y decisión APPROVED/APPROVED_WITH_WARNINGS/REJECTED según severidad y max_error_rate. Cada regla conserva rule_id cuando existe, code, columnas, severity, status, evaluated_count, failed_count, skipped_count, parámetros y condición. La evidencia identifica valor recibido y motivo. Una fila puede producir varios errores; error_rows cuenta registros afectados, no la suma de mensajes. WARNING conserva la fila aceptada y su advertencia. Una regla sin filas evaluadas no demuestra corrección técnica de una excepción.

El output reutilizable se guarda como Parquet INTAKE_ACCEPTED, con source_type=INTAKE_OUTPUT, canonical_artifact_id, parent_version_id y source_run_id. No tiene upload original ficticio. La UI muestra Artefacto derivado/Fuente de la versión y el linaje enlaza el input y el IntakeRun.

Caso operativo: aceptación y evidencia reutilizable

Para un archivo de pagos, required valida identificación, positive exige monto positivo y reference compara la cuenta con una versión del catálogo. El usuario declara cuando corresponda trim o decimal_parse; el motor nunca los añade por su cuenta. Se publica el contrato y se ejecuta sobre una versión concreta.

SUCCESS + REJECTED significa que la evaluación terminó y encontró incumplimientos; no debe repetirse como si hubiera ocurrido un error de infraestructura. La fila rechazada queda explicada por regla/valor/motivo. Las aceptadas generan una versión derivada trazable; su entrega posterior no necesita consultar de nuevo el archivo ni el catálogo remoto. Una regla WARNING no elimina la fila.

9. ReconOps y normalización declarada

ReconOps compara versiones inmutables de origen y destino. key_columns admite una o varias columnas. El control conserva key_normalization en su versión, config hash, manifest, plan y diagnostics.

El editor explica que las columnas clave identifican el mismo registro en ambos lados y que, al seleccionar varias, forman una clave compuesta cuyo orden se respeta. También aclara que la normalización se aplica antes de buscar coincidencias y no modifica los datasets originales. Las opciones visibles son “Conservar como están / Eliminar espacios externos”, “Conservar como están / Convertir a MAYÚSCULAS / Convertir a minúsculas” y “No normalizar / Normalización estándar (NFC) / Normalización de compatibilidad (NFKC)”. La ayuda Unicode explica la diferencia entre representación visual e interna.

```
{"trim": false, "case": "NONE", "unicode_normalization": "NONE"}
```

case admite NONE/UPPER/LOWER y unicode_normalization NONE/NFC/NFKC. Los controles nuevos usan NONE por defecto. Los históricos sin el campo conservan el comportamiento legacy de trim mediante adaptación versionada; no se cambia silenciosamente el resultado de runs anteriores.

Una vista previa Antes / Después utiliza hasta diez registros reales de origen y destino para mostrar la clave interna tras la normalización. En claves compuestas representa todas las columnas como conjunto ordenado y permite contrastar ambos lados. La muestra y el ejemplo dinámico se calculan en UI; no escriben datos ni alteran trim booleano, case (NONE, UPPER, LOWER) ni unicode_normalization (NONE, NFC, NFKC) persistidos.

Regla	Semántica
EXACT_COMPARE	Igualdad textual exacta tras la normalización declarada, con configuración independiente por par de columnas.
NUMERIC_TOLERANCE	Comparación Decimal con abs y percent opcional; denominador SOURCE/TARGET/MAX_ABS y política de denominador cero EXACT_ONLY. Umbral inclusivo.

Regla	Semántica
DATE_TOLERANCE	Diferencia temporal contra hours o days, con zona UTC y equal_nulls.
AGGREGATE_COMPARE	1:N agregando destino o N:1 agregando origen, con una o varias operaciones SUM/COUNT sobre el mismo lado. comparison_rules consumen las columnas resultantes.

EXACT_MATCH se conserva como alias explícito legacy de NUMERIC_TOLERANCE. Una tolerancia numérica cero sigue comparando magnitudes: "01" y "1" pueden coincidir. Esto es distinto de EXACT_COMPARE. amount_column/tolerance históricos se adaptan sin modificar el snapshot.

Políticas y orden de ejecución

Cada comparación admite null_policy MATCH_NULLS, MISMATCH o INVALID; el adaptador conserva equal_nulls de controles históricos. Con MATCH_NULLS dos null coinciden, uno solo produce diferencia; MISMATCH considera cualquier null una diferencia; INVALID clasifica el registro como inválido. Un valor numérico/temporal mal formado conserva INVALID y nunca se equipara a un null válido.

El orden determinista es transforms de origen/destino, normalización de claves, agrupación explícita y comparaciones. Se configuran las transformaciones por columna y lado; no existen conversiones implícitas adicionales. Los parámetros se persisten en control, config hash, diagnostics, manifest y Excel.

Las agregaciones nuevas sólo conservan claves y outputs definidos. Una comparación no puede tomar arbitrariamente el primer valor no agregado de un grupo. SUM inválido se propaga a la comparación que consume ese output; COUNT puede evaluarse independientemente. Los controles históricos que usaban aggregation singular se leen con su comportamiento anterior; una nueva publicación debe satisfacer las validaciones actuales. No se admite N:M ni fuzzy matching.

Clasificaciones estables

MATCH, VALUE_MISMATCH, SOURCE_ONLY, TARGET_ONLY, DUPLICATE_SOURCE, DUPLICATE_TARGET e INVALID. Las filas duplicadas no se convierten en coincidencias arbitrarias. Las comparaciones conservan valores, regla, diferencia, tolerancia y líneas origen/destino; agregaciones conservan las líneas de sus integrantes.

metrics incluye source_rows/target_rows, matched, mismatched, faltantes, duplicados, invalid, total_rows, match_rate, counts por clasificación y evaluadas/fallidas/inválidas por comparación. La decisión es CONFORME o WITH_FINDINGS. Los hallazgos agrupan categorías distintas de MATCH; UI y Excel permiten revisar el detalle verificable de cada comparación y el linaje de ambos inputs.

La precisión monetaria se implementa con Decimal y contexto suficiente para sumas, diferencias y porcentajes. Las expresiones tienen pruebas de paridad Polars/DuckDB donde se declara soporte. No se infiere equivalencia de todo el pipeline entre motores a partir de estas pruebas.

Caso operativo: claves, tolerancia y diferencias

Un control de conciliación puede fijar (cuenta, fecha) como clave compuesta y comparar importe con tolerancia Decimal. Antes de ejecutar se eligen los dos snapshots y se revisan transforms y null_policy. Una clave duplicada en origen no se empareja arbitrariamente con la primera fila destino; conserva la clasificación de duplicidad y su evidencia.

El resultado puede ser SUCCESS + WITH_FINDINGS. MATCH no implica igualdad de texto cuando se eligió tolerancia numérica; EXACT_COMPARE sí conserva la representación después de las transforms declaradas. Para temporales STRING, una prueba de igualdad no los convierte en instantes; DATE_TOLERANCE requiere su propio contrato temporal compatible.

10. Sentinel y métricas históricas

Sentinel ejecuta monitores sobre una DatasetVersion y produce checks explicables. Los controles legacy SCHEMA_REQUIRED, NULL_RATE, FRESHNESS y VOLUME_CHANGE permanecen compatibles. Una columna ausente se distingue de una columna presente con nulls.

En el editor, “Columnas requeridas” y “Columnas a revisar por nulos” son selectores múltiples alimentados por el esquema de la DatasetVersion elegida. Pueden conservar selecciones históricas mientras se publica una versión nueva; backend sigue validando que los nombres y la organización sean correctos.

Control	Parámetros / cálculo
SCHEMA_REQUIRED	Lista de columnas obligatorias del monitor.
NULL_RATE	Columnas observadas y máximo permitido.
FRESHNESS	Edad respecto a fecha de versión y max_age_hours.
VOLUME_CHANGE	Cambio porcentual respecto a versión anterior.
DISTINCT_COUNT	Columna y min/max de valores distintos.
DISTINCT_RATE	Columna y límites en fracción 0..1.
UNIQUENESS_RATIO	Proporción de registros únicos, con límites 0..1.
SCHEMA_TYPE	Tipo esperado o comparación con el esquema de la versión anterior.
METRIC_THRESHOLD	Métrica/columna, operador y threshold o min/max.
HISTORICAL_BAND	window, min_history, iqr_multiplier y fallback explícito.

Las reglas por registro de Intake también pueden evaluarse como checks de Sentinel con conteos evaluados/fallidos/excluidos. El resultado indica code, name, status, actual, expected, severity, rule_id cuando existe y message. La severidad declarada llega a Findings y Excel. La salud es la fracción de checks aprobados; cero fallos da HEALTHY y cualquier fallo da ALERT, incluso cuando su severidad es WARNING.

Compatibilidad de series

SentinelMetricHistory conserva run_id, monitor_id, metric_key, dimensiones y hash, numeric_value, method, metric_definition_version y observed_at. Sólo se comparan ejecuciones SUCCESS con método/versión compatibles. El historial no mezcla perfiles legacy normalizados con observaciones nuevas exactas.

La banda histórica utiliza mediana e IQR con el método MEDIAN_IQR_LINEAR_V1. Cuando no hay historia suficiente, el resultado identifica FIXED_THRESHOLD_FALLBACK o PREVIOUS_COMPATIBLE_METRIC según la política. El expected muestra qué método se aplicó; no inventa una referencia estadística.

El monitor puede detectar schema drift sin que el worker falle técnicamente. El Run original conserva ALERT aunque una versión posterior vuelva a estar saludable. Las excepciones asociadas sólo se validan cuando una ejecución posterior del mismo snapshot de monitor termina HEALTHY.

Programación local e historia visible

El worker consulta monitor_schedules antes de consumir la cola existente. La programación referencia una Configuration inmutable. Una edición crea MonitorScheduleVersion con intervalo, activación, inicio y actor; no mueve programaciones a una nueva versión del monitor automáticamente. Se admiten intervalos de un minuto a 31 días. La UI recibe hora local y la convierte a UTC; la API exige zona explícita.

En cada despacho se fija la última DatasetVersion registrada. No se consulta ni actualiza automáticamente la fuente externa; el refresh de una conexión sigue siendo adquisición separada. MonitorOccurrence conserva configuración, revisión de horario, DatasetVersion, fecha prevista, despacho y Run. El Run aporta inicio real, finalización, estado, métricas y evidencia. La metadata schedule se incluye en execution_plan y manifest.

COALESCE_LATEST agrupa intervalos vencidos tras una pausa; SKIP_WHILE_ACTIVE evita solapar runs del mismo monitor. NO_DATASET_VERSION documenta la ausencia de entrada. Cursor, ocurrencia, Run, Job y auditoría se confirman en una sola transacción con compare-and-swap y unicidad por fecha prevista. El actor es SYSTEM / trackvance:local-scheduler.

Mientras Docker o el worker están detenidos no hay ejecución. Un job largo puede retrasar el siguiente tick; la diferencia entre fecha prevista e inicio real queda visible. Las series agrupan muestras por métrica, dimensiones, método y versión, con un máximo de 2000 muestras recientes. Las alertas internas son Findings y aparecen en Sentinel y Centro de Control. NotificationDelivery prepara una interfaz futura; Email, Teams, Slack y Webhook no están integrados.

Caso operativo: monitoreo y comparación histórica

Un monitor fija columnas, tasas permitidas y checks. Ejecutarlo contra un nuevo snapshot conserva ambos IDs y la definición exacta de cada métrica. Una alerta de null rate conduce al detalle del Run y al Finding, no modifica la versión observada. Deshabilitar una programación impide próximos despachos, sin borrar ocurrencias ni series ya registradas.

Al cambiar la definición de una métrica no se mezclan valores incompatibles bajo una sola tendencia. El lector agrupa por método/versión/dimensiones. En 0.5.1 los temporales sin zona siguen siendo STRING en los checks de schema; la apariencia de fecha en una celda no autoriza reinterpretar la columna.

11. Excepciones con validación técnica

Una excepción nace de un Finding y conserva finding_id, run_id de origen y configuration_id del snapshot que lo produjo. El Run de origen nunca cambia su decisión porque un caso se cierre o los datos se corrijan.



Estado	Función
OPEN / Abierta	Hallazgo convertido en caso, pendiente de gestión.
ASSIGNED / Asignada	Caso asignado a un usuario activo y autorizado de la organización.
INVESTIGATING / En gestión	Responsable investiga causa y prepara corrección.
PENDING_VALIDATION	Espera evidencia técnica posterior.
RESOLVED / Resuelta	Evidencia válida, causa y resolución documentadas.
DISCARDED / Descartada	Cierre administrativo con motivo obligatorio.
ACCEPTED / Aceptada	Riesgo o situación aceptada con motivo obligatorio.
NOT_APPLICABLE / No aplica	Cierre administrativo justificado.
REOPENED / Reabierta	Caso cerrado que vuelve a gestión con comentario y nueva fecha de referencia.

Criterios de elegibilidad

Se requiere un Run posterior SUCCESS, de la misma organización y el mismo configuration_id. Publicar otra versión de configuración crea otro snapshot y no valida automáticamente la excepción anterior. Se selecciona la candidata elegible más reciente; no se acepta una validación positiva obsoleta cuando existe una ejecución posterior que vuelve a fallar.

- Intake: la regla con rule_id que originó el hallazgo debe estar presente, evaluar al menos una fila y tener cero incumplimientos. En un caso histórico abierto sin ID se identifica la regla mediante su código/columna y fingerprint compatible, pero también se exigen métricas suficientes: evaluated_count mayor que cero y failed_count igual a cero. La mera ausencia de otro Finding no demuestra que la regla se haya evaluado.
- ReconOps: el control debe quedar conforme o desaparecer la clasificación asociada al hallazgo.
- Sentinel: el monitor debe volver a HEALTHY.

La UI deshabilita Resolver mientras no se cumplen los criterios y muestra el motivo. Una validación correcta se presenta como Validada técnicamente y enlaza el Run confirmatorio. Resolver sólo se acepta desde PENDING_VALIDATION y exige causa raíz y resolución. La API reevalúa antes de guardar.

El worker actualiza evidencia de casos PENDING_VALIDATION al terminar un Run. Conserva validation_run_id, validated_at, validation_evidence y evento TECHNICAL_VALIDATION. auto_resolve_enabled es una política por caso deshabilitada por defecto; habilitarla requiere permiso de cierre. Cuando está activa y la misma comprobación técnica es válida, el worker puede resolver el caso con actor SYSTEM, evento EXCEPTION_AUTO_RESOLVED, política CASE_OPT_IN y Run confirmatorio. No inventa una causa raíz humana y no duplica cierres en reintentos.

Gestión operativa

Cada caso admite assigned_user_id, prioridad, SLA en horas, due_at, comentarios y adjuntos. La prioridad es independiente de la severidad histórica. El SLA se calcula desde creación o reapertura, con UTC; una fecha objetivo explícita prevalece. No hay calendario laboral implícito. Los filtros backend cubren estado, módulo, severidad, prioridad, responsable, vencimiento y texto.

Reabrir exige comentario; conserva el cierre anterior en timeline y reinicia el plazo. Una ejecución creada antes de la reapertura no prueba la corrección actual. Se preservan las transiciones históricas OPEN -> INVESTIGATING y reapertura hacia OPEN para compatibilidad.

Los adjuntos permitidos se limitan a 10 MiB, se publican inmutables mediante StorageProvider y registran Artifact EXCEPTION_ATTACHMENT, hash y relación EXCEPTION_EVIDENCE. No se ejecutan ni se usan como reglas. La descarga exige organización/permisos, verifica integridad, fuerza attachment/nosniff y genera auditoría. Nombre y descripción se sanitizan; el filename nunca define una ruta física.

Las mutaciones usan version para concurrencia optimista. Timeline conserva actor estable, fecha, estado anterior/nuevo, comentario y referencia de validación. Los cierres administrativos no equivalen a RESOLVED ni fabrican evidencia. WAITING_EXTERNAL/FALSE_POSITIVE y resoluciones históricas permanecen legibles; los registros anteriores sin prueba se identifican como históricos, sin inventar un Run confirmatorio.

Si el Run candidato no contiene métricas suficientes para identificar y probar la regla histórica, el nuevo cierre queda bloqueado con motivo de evidencia insuficiente. Se requiere ejecutar nuevamente el mismo control y obtener evidencia verificable. Esto no modifica casos ya RESOLVED ni sus timelines históricos; endurece únicamente las nuevas decisiones de resolución.

Caso operativo: cierre respaldado por nueva ejecución

Asignar una excepción a una persona no cambia la severidad del Finding original. El responsable documenta causa, corrección y evidencia; PENDING_VALIDATION espera un nuevo Run elegible de la misma configuración. Una ejecución anterior a la corrección o una regla con cero filas evaluadas no demuestra resolución.

Resolver conserva vínculo al Run confirmatorio. Descartar, aceptar o marcar no aplicable es un cierre administrativo con motivo, no una aprobación técnica. Una reapertura deja historia y exige nueva validación posterior. Estos estados son distintos de la revisión UNKNOWN de Delivery: no se recicla Exception como sustituto de su registro operacional específico.

12. Centro de Control y navegación

El Centro de Control es un cockpit operativo calculado por backend. El usuario puede identificar qué falla, qué requiere atención y abrir la acción correspondiente. Las cifras no son constantes de presentación.

Elemento	Comportamiento actual
Filtros globales	Período 7d/30d/90d/all, dataset, módulo, estado operativo y criticidad.
Indicadores superiores	Salud general, controles fallidos, excepciones abiertas y datasets afectados. Variación contra período anterior cuando existe una comparación válida.
Requiere tu atención	Excepciones, hallazgos y fallos priorizados; muestra dataset, módulo, problema, fecha y acción directa.
Salud de los datos	Evolución temporal y desglose Intake/Recon/Sentinel.
Datasets afectados	Salud, hallazgos, última ejecución y tendencia.
Resumen por módulo	Ejecuciones/controles/monitores con fallos y alertas.
Ejecuciones recientes	Dataset, registros, hallazgos y duración cuando están disponibles; estado técnico separado de decisión.
Accesos rápidos	Carga de dataset y ejecución de operaciones.

health_score se pondera por unidades evaluadas según la agregación del backend. Las variaciones devuelven previous/delta o null cuando no hay base válida. Los filtros respetan organización y los enlaces navegan al recurso correspondiente. No se presenta ausencia de historia como mejora o deterioro.

Rutas SPA

/; /datasets; /datasets/:id; /connections; /delivery/*; /intake/*; /recon/*; /sentinel/*; /runs; /runs/:id; /exceptions; /rules; /audit; /settings/*. Delivery incluye listado, /delivery/new, destinos y detalle de destino. La administración de usuarios está integrada en Configuración. El menú conserva los módulos y el diseño navy/teal. Estados de carga, vacío y error tienen tratamiento específico; se conserva request_id en errores para diagnóstico.

Cerrar sesión confirma /auth/logout, limpia token CSRF, caché de consultas y estado de sesión, y navega con reemplazo a /. El usuario vuelve a la pantalla inicial de Trackvance (incluido “Entrar al entorno demo” cuando esté habilitado) sin mantener una pantalla autenticada en el historial inmediato.

Los detalles de dataset separan archivo original, fuente derivada, esquema/perfil y linaje. Los detalles de run separan Completada de Rechazado/Con hallazgos/Alerta. Los labels de negocio acompañan reason codes técnicos en resultados y evidencia. La numeración se muestra como Línea del archivo cuando representa línea física o Registro de la versión cuando corresponde.

Carga diferida y continuidad de la navegación

React.lazy y Suspense separan las rutas pesadas sin introducir librerías. El shell, identidad y navegación permanecen disponibles mientras se descarga el módulo. El detalle Delivery se carga por separado para evitar que Runs importe todo el builder de salida. La frontera de error ofrece recuperación visible si falla la carga de un chunk; no deja una región vacía como éxito aparente.

La medición 0.5.1 reduce el JavaScript inicial de 611,41 kB/179,47 kB gzip a 364,97 kB/116,77 kB gzip: aproximadamente 40,3% y 34,9% respectivamente. Es una reducción del arranque, no de la suma de todos los assets; las features siguen descargándose al usarlas. El build deja de emitir la advertencia de chunk mayor que 500 kB. Las rutas directas, recarga y permisos se prueban en navegador.

13. StorageProvider, artefactos y linaje

StorageProvider es el puerto de almacenamiento interno. `temporary_path` asigna staging local temporal; `put_file` publica bytes y metadata inmutables; `materialize/materialize_reference` obtiene un archivo verificable para el motor local; `open_read/exists` completan el acceso. Un adaptador remoto deberá administrar su cache materializado y su ciclo de limpieza.

FileArtifactStore confina las rutas al root configurado, verifica tamaño/SHA-256 y publica sin reemplazar bytes históricos. El nombre recibido nunca se usa como path arbitrario. Los servicios nuevos no construyen el layout del volumen; el adaptador conserva registro directo/promote para compatibilidad y backfill.

Kind	Uso
ORIGINAL_UPLOAD	Archivo realmente recibido del usuario.
CANONICAL_PARQUET	Representación canónica de una versión cargada.
INTAKE_ACCEPTED	Parquet de filas aceptadas; base reutilizable de versión derivada.
INTAKE_ERRORS	Detalle de incumplimientos Intake.
RECON_RESULTS	Resultados detallados de conciliación.
SENTINEL_PROFILE	Checks/resultados del monitor.
RUN_MANIFEST	Evidencia JSON de ejecución.
DELIVERY_RECEIPT	Recibo de commit remoto confirmado, sin secretos ni filas completas.
EXPORT_XLSX	Reporte estructurado generado bajo demanda.
EXCEPTION_ATTACHMENT	Evidencia documental adjunta a un caso, inmutable y con hash.
GENERATED_DEMO / legacy	Datos ficticios o artefactos históricos identificados explícitamente.

Artifact guarda `id`, `organization_id`, `kind`, `name`, `path` interno, `sha256`, `size_bytes`, `media_type` y `fecha`. ArtifactLink registra relaciones dirigidas `DERIVED_FROM`, `RUN_INPUT`, `RUN_OUTPUT`, `INTAKE_ACCEPTED_FROM`, `RUN_REFERENCE`, `EXCEPTION_EVIDENCE`, `SOURCE_SNAPSHOT`, `REFRESH_OF`, `DELIVERY_INPUT`, `DELIVERED_TO`, `DELIVERY_RECEIPT`, `EVIDENCE_OF` y `EXPORT_OF` entre versiones, runs, fuentes, destinos, intentos y artefactos.

Antes de procesar o descargar, se comprueba la identidad registrada y su integridad.

ARTIFACT_HASH_MISMATCH/ARTIFACT_INTEGRITY_ERROR impide usar bytes corruptos; no recalcula silenciosamente el hash histórico. El backfill aditivo registra archivos legacy verificables, cuenta ausentes/corruptos y conserva historia.

Una publicación interrumpida puede dejar bytes huérfanos no referenciados después de un rollback. Esos bytes no reemplazan artefactos válidos. Retención, recolección de huérfanos y políticas de object storage son evolución pendiente y deben implementarse con referencias transaccionales y auditoría.

14. Manifest schema 2 y auditoría

Cada Run completado publica un manifest inmutable con `schema_version=2`. Excepción operativa explícita: si una entrega ya COMMITTED pierde la publicación local, conserva el commit y marca `evidence_status=PENDING_REPAIR`; nunca repite la transacción para fabricar la evidencia. `read_manifest/adapt_manifest` centraliza la lectura de v1 histórico; la adaptación se realiza en memoria, sin sobrescribir archivos anteriores.

```
{
  "schema_version": 2,
  "run_id": "<id>",
  "module": "recon",
  "initiated_by": {
    "type": "USER", "id": "<id estable>",
    "display_name": "Equipo Trackvance"
  },
  "result_artifacts": [{
    "artifact_id": "<id>", "kind": "RECON_RESULTS",
    "name": "results.parquet", "sha256": "<hash>",
    "size_bytes": 1234
  }]
}
```

El contrato completo incluye `started_at/finished_at`, identidad/legacy, versión de Trackvance, plan processing con engine/version y schedule cuando corresponde, inputs con DatasetVersion/SHA/schema hash/origen/ingestion_metadata, configuración efectiva/id/version/config_hash/stored_config_hash, métricas, `output_version_id` y `result_artifacts`. references añade identidad y hashes de DatasetVersions usadas para integridad referencial. Delivery añade una sección de `delivery` con DestinationVersion, target, estrategia, intento, resultado y receipt; es una extensión aditiva de schema 2. Los snapshots anteriores no se reescriben.

La versión del producto y la del motor se distinguen: `engine_version` de nivel superior identifica Trackvance en el manifest actual; `processing.engine_version` identifica la versión real de Polars. La UI y exports muestran ambas con su contexto.

Identidad y eventos

AuditEvent conserva `actor_type`, `actor_id` estable, actor visible, `actor_legacy`, `event_type`, `subject_type/id`, `request_id`, `run_id`, metadata sanitizada y timestamp. Actor puede representar USER, WORKER o SYSTEM. Los nombres visibles no sustituyen la identidad persistente.

DATASET_UPLOADED, DATASET_DERIVED, CONFIGURATION_PUBLISHED, RUN_QUEUED, RUN_COMPLETED, EXCEPTION_CREATED, EXCEPTION_UPDATED y EXCEPTION_VALIDATION_CHECKED documentan el ciclo. EXPORT_DOWNLOADED, EVIDENCE_DOWNLOADED y ARTIFACT_DOWNLOADED registran el acceso relevante a evidencia. La UI enlaza recursos desde los eventos.

La administración añade USER_CREATED, USER_UPDATED y USER_PASSWORD_RESET. La gestión operativa audita comentarios, adjuntos y resolución automática; la programación conserva revisiones y eventos de despacho. Delivery registra publicación, encolado, STARTED, COMMITTED, FAILED o UNKNOWN con actor estable y referencias, nunca contraseña ni filas. Las filas de negocio recibidas, credenciales, hashes de contraseña y sesiones nunca forman parte del payload público de estos eventos.

Se excluyen claves de passwords, tokens y secretos de metadata/evidencia. Los casos legacy sin identidad demostrable se marcan como legacy; no se atribuyen a un UUID de usuario inventado. Las auditorías de exportación agregan evidencia sin alterar el manifest original del Run.

15. Reportes Excel de negocio

IMPLEMENTADO. La UI ofrece Exportar Excel y genera .xlsx mediante openpyxl. CSV permanece únicamente como endpoint deprecated para consumidores históricos. El Excel es un artefacto derivado; Parquet sigue siendo canónico.

Módulo	Hojas y contenido
Intake	Resumen: contrato/version, dataset/version, run, fechas, estado/decisión, filas/tasas/umbral/motor y reglas con evaluadas/fallidas/excluidas/severidad. Errores: línea, regla, columnas, valores recibidos, severidad, motivo y clasificación. Reglas: configuración efectiva. Trazabilidad: inputs, referencias, IDs/hashes/actor/plan/artefactos.
ReconOps	Resumen: control, ambos inputs, claves/comparaciones/tolerancias/nulls/transforms/agregaciones, siete conteos y match rate. Resultados: clave, clasificación, valores, diferencia, tolerancia, líneas, mensaje y comparaciones. Hallazgos: no MATCH. Trazabilidad: ambos inputs y evidencia.
Sentinel	Resumen: monitor, versión, filas, checks/fallos, salud y decisión. Controles: código/estado/observado/esperado/detalle, severidad y conteos evaluados/fallidos/excluidos. Hallazgos: sólo fallidos. Trazabilidad: input, referencias, configuración, motor y artefactos.

El diseño usa encabezados navy, teal para acentos, rojo para incumplimientos y amber para warning. Tablas con autofiltro, paneles inmovilizados, anchos legibles, ajuste de texto y formatos de fecha, porcentaje y decimal. Las columnas técnicas se concentran en Trazabilidad y los reason codes siguen disponibles.

Tipos y seguridad

Identificadores se escriben como texto. Decimales de más de 15 cifras significativas también se conservan como texto para evitar pérdida de precisión de Excel. Los valores de negocio que comienzan con =, +, -, @, tabulación o espacios nunca se convierten en fórmula ejecutable ni link externo. Las reglas de formato condicional son constantes internas.

No se trunca evidencia silenciosamente: celdas mayores que 32767 caracteres o tablas que exceden límites de Excel producen EXPORT_LIMIT_EXCEEDED. Controles XML inválidos se escapan de forma visible. Passwords/tokens/secrets no se incluyen.

GET /runs/{id}/export.xlsx exige sesión, exports:download, organización, SUCCESS e integridad de manifest/resultados. El MIME es application/vnd.openxmlformats-officedocument.spreadsheetml.sheet; Content-Disposition usa trackvance_<module>_<run>.xlsx sanitizado y X-Artifact-ID identifica el export. Se registra EXPORT_XLSX/EXPORT_OF y auditoría.

16. Ejecución, planificación y recuperación

POST de ejecución valida configuración/versions, fija actor e inputs inmutables, calcula preflight y crea Run más Job en la misma sesión. Idempotency-Key evita crear runs duplicados para el mismo endpoint/organización/key/payload; cambiar el payload con la misma key produce IDEMPOTENCY_CONFLICT.

Estado técnico	Significado
QUEUED	Trabajo persistido esperando worker.
RUNNING	Worker posee lease y procesa la ejecución.
SUCCESS	Procesamiento terminado; en Delivery, commit remoto confirmado. La evidencia local se publica después y puede quedar marcada PENDING_REPAIR sin repetir la transacción.
FAILED	Error técnico durante ejecución.
UNKNOWN	Sólo Delivery: resultado remoto no confirmable; no equivale a fallo/commit y bloquea reintento automático.
FAILED_PRECONDITION	Preflight no permite ejecutar; no se simula resultado.

Estado técnico	Significado
CANCELLED	Cancelación atendida por el worker.

DatabaseJobQueue publica Jobs con run_id único y lane DEFAULT o DELIVERY. Cada worker reclama únicamente su lane con actualización condicional, identifica propietario y lease_until, registra intentos y recupera leases vencidos. Heartbeats separados informan actividad local y salud. El scheduler sólo corre en DEFAULT. Delivery adquiere primero el Run y después el Job, reclama el lease antes de reconciliar y vuelve a validarlo antes de cambiar estados o publicar. PreparedDelivery completa validación y materialización local antes de STARTED. La cancelación que gana antes de STARTED impide la escritura; después de STARTED prevalece un resultado remoto conocido. Un STARTED recuperado o lease perdido sin confirmación queda UNKNOWN, no se reejecuta y no sobrescribe evidencia histórica.

ExecutionPlanner

Estima memoria de trabajo desde bytes/filas/columnas y un factor mayor para Recon. Comprueba disco temporal libre. Los defaults son 512 MiB de presupuesto blando, 16 MiB de reserva mínima de disco y 300 segundos en resource_budget. Estos valores son configurables por entorno.

El pipeline real es Polars/Python. Si la estimación requiere PySpark, el plan rechaza con ENGINE_UNAVAILABLE; disco insuficiente produce RESOURCE_DISK_INSUFFICIENT. No hay fallback silencioso ni adapter Spark instalado. DuckDB se usa en metadata y paridad de expresiones, no como pipeline completo seleccionable.

La presencia de timeout_seconds en el plan no equivale a un límite duro por proceso. La interrupción bajo carga, kill/restart de procesos y el presupuesto real de memoria requieren certificación de estrés adicional. La cancelación es cooperativa; los lanes DEFAULT/DELIVERY están implementadas, mientras Redis/Celery, prioridades generales y workers distribuidos son futuros.

17. Modelo persistente y migraciones

PostgreSQL mantiene 25 tablas de aplicación, además del control de revisión de Alembic. El esquema real es más compacto que el ERD objetivo inicial: organización se expresa mediante organization_id lógico y roles mediante users.role más política central. No existen tablas dedicadas organizations/roles/permissions; la administración de los cinco roles base está implementada sobre el modelo actual.

Tabla	Identidad / relación principal
users	Usuario, organization_id, role, password_hash, active, version y fechas de actualización/cambio de contraseña.
sessions	Token hash, usuario, CSRF y expiración.
datasets	Nombre único por organización, área, owner, criticidad y estado.
dataset_versions	Versión única por dataset, schema/profile, artifacts y lineage.
configurations	Snapshot por módulo; previous_version_id y datasets.
runs	Snapshot de inputs/config, actor, plan, métricas, decisión y evidencia.
jobs	Run único, lane DEFAULT/DELIVERY, estado, intentos y lease.
delivery_destinations	Identidad de destino, organización, motor, revisión vigente, estado y última prueba.
delivery_destination_versions	Configuración inmutable, referencia de secreto y config_hash.
delivery_attempts	Run/destino, número/clave idempotente, estado remoto, target, métricas, referencia/error y fechas.
findings	Run, código, fingerprint único por Run y detalles.
exceptions	Finding único, configuración, origen, validación, asignación, prioridad, SLA, reapertura, política automática y timeline/version.
exception_attachments	Caso, artifact inmutable, autor y descripción.
audit_events	Actor estable, sujeto, request/run y metadata sanitizada.

Tabla	Identidad / relación principal
idempotency_keys	Organización/ruta/key, hash request y Run.
artifacts	Identidad inmutable, kind, locator, hash/tamaño/MIME.
artifact_links	Relación dirigida única entre entidades.
metric_history	Métrica/dimensión/run, método/version y valor.
external_connections	Identidad, revisión vigente, estado y última prueba; ámbito por organización.
external_connection_versions	Configuración inmutable, referencia de secreto y config_hash.
dataset_source_bindings	Dataset, conexión, schema, tabla/vista y overrides.
monitor_schedules	Monitor único, revisión vigente, habilitación, cursor y fecha de actualización.
monitor_schedule_versions	Revisión inmutable del intervalo, inicio, activación y actor de una programación.
monitor_occurrences	Programación/revisión, monitor, versión evaluada, fecha prevista/real, estado, motivo y Run.
delivery_reviews	Run/attempt UNKNOWN, revisor estable y nombre, outcome, nota, fecha de verificación y creación.

Cadena de trazabilidad

Conexión/revisión -> Dataset -> DatasetVersion -> Configuration -> Run -> Finding -> Exception. En Delivery, DatasetVersion + DestinationVersion -> Configuration DELIVERY -> Run -> DeliveryAttempt -> receipt/manifest. La excepción añade Validation Run de la misma Configuration y artifacts adjuntos. La programación vincula su revisión y fecha prevista al Run; las reglas referenciales añaden snapshots de consulta. ArtifactLink conecta originales, canónicos, outputs, manifests, receipts y exports. Las claves de organización y comprobaciones del servicio acotan todos los accesos.

Migración	Cambio
0001_initial	Esquema original.
0002_evidence_v2	Identidad, artefactos, linaje, evidencia y métricas versionadas.
0003_dataset_ingestion_metadata	Formato/lector/opciones/esquema nativo por versión.
0004_exception_validation	Configuración de origen, validación técnica y cierre administrativo.
0005_external_connections	Conexiones externas, configuraciones versionadas y bindings de datasets.
0006_local_identity_exceptions	Versionado de usuarios, fechas, campos operativos de excepciones y exception_attachments.
0007_monitor_scheduling	Programaciones, revisiones inmutables y ocurrencias Sentinel; unicidad e índices de despacho.
0008_data_delivery	Destinos/revisiones/intentos de entrega, lane persistida e índices/constraints de Data Delivery.
0009_delivery_reviews	Tabla append-only de revisiones operacionales UNKNOWN; FKs, índices y CHECK de outcomes.

La revisión vigente es 0009_delivery_reviews. Todas las evoluciones son migraciones aditivas nuevas; no se modifican migraciones aplicadas. API startup aplica Alembic y backfill idempotente. SQLite legacy sólo se adopta cuando coincide con un schema reconocido; no se fuerza sobre estructuras desconocidas. Downgrade/upgrade se prueban en bases temporales, nunca sobre datos operativos.

18. API y errores

La base es /api/v1. El runtime FastAPI y backend/API_CONTRACT.md describen los campos exactos implementados. El inventario siguiente se genera desde backend/openapi.json; el snapshot 0.5.1 añade reparación y revisión operacional al inventario de Data Delivery. La tabla siguiente forma parte del inventario ejecutable revisado antes de construir el PDF.

Método	Ruta	Operación
GET	/health	Health
GET	/health/ready	Ready
GET	/api/v1/health	Health
GET	/api/v1/health/ready	Ready
POST	/api/v1/auth/demo	Demo Login
POST	/api/v1/auth/login	Login
GET	/api/v1/me	Me
POST	/api/v1/auth/logout	Logout
GET	/api/v1/datasets	Datasets
POST	/api/v1/datasets	New Dataset
POST	/api/v1/datasets/uploads/inspect	Inspect Dataset Upload
GET	/api/v1/datasets/{dataset_id}	Dataset Detail
GET	/api/v1/datasets/{dataset_id}/schema	Dataset Schema
POST	/api/v1/datasets/{dataset_id}/versions/upload	Upload Version
GET	/api/v1/dataset-versions/{version_id}/profile	Profile
GET	/api/v1/monitors	Configurations
POST	/api/v1/monitors	Monitor
GET	/api/v1/recon/controls	Configurations
POST	/api/v1/recon/controls	Recon Control
GET	/api/v1/intake/contracts	Configurations
POST	/api/v1/intake/contracts	Intake Contract
POST	/api/v1/intake/runs	Intake Run
POST	/api/v1/recon/runs	Recon Run
POST	/api/v1/monitors/{monitor_id}/runs	Monitor Run
GET	/api/v1/runs	Runs
GET	/api/v1/runs/{run_id}	Run Detail
POST	/api/v1/runs/{run_id}/cancel	Cancel Run
GET	/api/v1/recon/runs/{run_id}/results	Results
GET	/api/v1/intake/runs/{run_id}/errors	Results
GET	/api/v1/runs/{run_id}/results	Results
GET	/api/v1/runs/{run_id}/evidence	Evidence
GET	/api/v1/artifacts/{artifact_id}/download	Artifact Download

Método	Ruta	Operación
GET	/api/v1/runs/{run_id}/export.xlsx	Export Excel
GET	/api/v1/runs/{run_id}/export.csv	Export Csv (legacy)
GET	/api/v1/monitors/{monitor_id}/metrics	Monitor Metrics
GET	/api/v1/findings	Findings
POST	/api/v1/findings/{finding_id}/exceptions	Finding Exception
GET	/api/v1/audit-events	Audit Events
GET	/api/v1/rules	Rules
GET	/api/v1/system/engines	Engines
GET	/api/v1/dashboard	Dashboard
POST	/api/v1/monitors/{configuration_id}/versions	Publish Configuration Version
POST	/api/v1/recon/controls/{configuration_id}/versions	Publish Configuration Version
POST	/api/v1/intake/contracts/{configuration_id}/versions	Publish Configuration Version
GET	/api/v1/runs/{run_id}/execution-plan	Run Execution Plan
GET	/api/v1/runs/{run_id}/diagnostics	Run Diagnostics
POST	/api/v1/execution-plans/preview	Preview Execution
GET	/api/v1/connections	List Connections
POST	/api/v1/connections	Create Connection
POST	/api/v1/connections/test	Test Draft
GET	/api/v1/connections/{connection_id}	Connection Detail
PATCH	/api/v1/connections/{connection_id}	Update Connection
DELETE	/api/v1/connections/{connection_id}	Delete Connection
POST	/api/v1/connections/{connection_id}/test	Test Connection
GET	/api/v1/connections/{connection_id}/schemas	Schemas
GET	/api/v1/connections/{connection_id}/objects	Objects
GET	/api/v1/connections/{connection_id}/preview	Preview
POST	/api/v1/connections/{connection_id}/datasets	Register Source Dataset
POST	/api/v1/datasets/{dataset_id}/refresh-source	Refresh Source
GET	/api/v1/delivery/destinations	List Destinations
POST	/api/v1/delivery/destinations	Create Destination
POST	/api/v1/delivery/destinations/test	Test Destination Draft
GET	/api/v1/delivery/destinations/{destination_id}	Destination Detail
PATCH	/api/v1/delivery/destinations/{destination_id}	Update Destination
DELETE	/api/v1/delivery/destinations/{destination_id}	Delete Destination
POST	/api/v1/delivery/destinations/{destination_id}/test	Test Destination
GET	/api/v1/delivery/destinations/{destination_id}/schemas	Destination Schemas
GET	/api/v1/delivery/destinations/{destination_id}/tables	Destination Tables
GET	/api/v1/delivery/destinations/{destination_id}/table-metadata	Destination Table Metadata

Método	Ruta	Operación
POST	/api/v1/delivery/preview	Delivery Preview
POST	/api/v1/delivery/preflight	Delivery Preflight
GET	/api/v1/delivery/configurations	List Delivery Configurations
POST	/api/v1/delivery/configurations	Publish Delivery Configuration
POST	/api/v1/delivery/configurations/{configuration_id}/versions	Publish Delivery Configuration Version
GET	/api/v1/delivery/runs	List Delivery Runs
POST	/api/v1/delivery/runs	Create Delivery Run
GET	/api/v1/delivery/runs/{run_id}/attempts	Delivery Attempts
GET	/api/v1/delivery/runs/{run_id}/receipt	Delivery Receipt
POST	/api/v1/delivery/runs/{run_id}/repair-evidence	Repair Evidence
GET	/api/v1/delivery/runs/{run_id}/reviews	Delivery Reviews
POST	/api/v1/delivery/runs/{run_id}/reviews	Review Delivery Outcome
GET	/api/v1/users	Users
POST	/api/v1/users	Create User
GET	/api/v1/users/roles	Roles
GET	/api/v1/users/{user_id}	User Detail
PATCH	/api/v1/users/{user_id}	Edit User
POST	/api/v1/users/{user_id}/reset-password	Reset Password
GET	/api/v1/exceptions	Exceptions
GET	/api/v1/exceptions/assignees	Assignees
GET	/api/v1/exceptions/{case_id}	Detail
PATCH	/api/v1/exceptions/{case_id}	Update Case
POST	/api/v1/exceptions/{case_id}/validate	Validate
POST	/api/v1/exceptions/{case_id}/comments	Comment
POST	/api/v1/exceptions/{case_id}/attachments	Upload Attachment
GET	/api/v1/exceptions/{case_id}/attachments/{attachment_id}/download	Attachment Download
GET	/api/v1/monitors/{monitor_id}/schedule	Get Schedule
POST	/api/v1/monitors/{monitor_id}/schedule	Configure Schedule
GET	/api/v1/monitors/{monitor_id}/occurrences	Occurrences
GET	/api/v1/monitors/{monitor_id}/series	Metric Series
GET	/api/v1/monitors/{monitor_id}/alerts	Monitor Alerts

19. Contratos HTTP y permisos

Las listas comunes devuelven {items,total}; IDs son strings opacos y fechas ISO UTC. El cockpit devuelve un DTO agregado. Los downloads usan el MIME del artefacto verificado y filename sanitizado.

Autenticación utiliza cookie HttpOnly trackvance_session. Mutaciones autenticadas requieren X-CSRF-Token y validación de origen. Sesiones duran 12 horas en el prototipo. Passwords se verifican mediante Argon2. El acceso demo es explícito, depende de DEMO_ACCESS_ENABLED y no representa SSO de producción. DEMO_SEED_ENABLED controla por separado la creación de datos sintéticos.

Rol	Alcance local
Administrator	Lectura, autoría, ejecución, exports, auditoría, sistema, cierre de excepciones y administración/uso de conexiones.
Data Owner / Lead	Lectura/autoría/ejecución, exports, auditoría, cierre y administración/uso de conexiones.
Data Analyst	Lectura/autoría/ejecución, exports, auditoría y uso de conexiones; sin administración de conexiones ni cierre de excepciones.
Operations	Lectura/exports y gestión de excepciones según permiso; solo consulta metadata de conexiones.
Auditor	Lectura/exports/auditoría/usuarios/sistema y metadata de conexiones; sin mutaciones operativas.

permissions.py es la política central; roles desconocidos no reciben permisos. Se distinguen datasets:read/write, configurations:write, runs:read/execute, exceptions:read/write/close, connections:read/use/manage, exports:download, artifacts:download, audit:read, users:read/write y system:read. connections:read permite consultar metadata; use permite probar una conexión guardada, explorar y adquirir snapshots; manage permite crear, editar, deshabilitar, eliminar y probar borradores. Los cierres técnicos y administrativos requieren exceptions:close además del permiso de gestión.

Data Delivery reutiliza provisionalmente esos permisos: destinos usan connections:read/manage/use; preview/preflight y publicación usan configurations:write; consulta/ejecución usan runs:read/execute; receipt usa artifacts:download. No existen aún permisos delivery:* ni gobierno por destino/estrategia. Ese RBAC granular permanece explícitamente pendiente.

Administración local implementada

Administrator puede listar, crear, editar, activar/desactivar usuarios, asignar roles base y restablecer contraseña. Auditor consulta cuentas y permisos efectivos; los demás accesos dependen de users:read/write. Las bajas son desactivaciones para conservar referencias históricas. Data Owner es un alias compatible de Data Owner / Lead; no se admiten roles arbitrarios.

La contraseña explícita usa SecretStr, de 12 a 1024 caracteres, no se recorta y sólo persiste como hash Argon2. El reset define una nueva contraseña y revoca todas las sesiones; cambiar correo, rol o actividad también las revoca. Las respuestas nunca incluyen hash, token, sesión ni contraseña. El administrador entrega la contraseña por un canal seguro fuera de Trackvance.

Las mutaciones requieren version y compare-and-swap. La degradación o desactivación de administradores protege al último administrador activo de la organización mediante bloqueo transaccional en PostgreSQL. Los conflictos de correo no revelan organizaciones ajenas. El middleware verifica rol y actividad actuales en cada solicitud. OIDC/SSO y directorio multi-organización administrable continúan como objetivos posteriores.

Errores operativos

El envelope es {error:{code,message,details,request_id}}. Sesión inválida devuelve 401; permiso/CSRF 403; recurso no accesible 404; conflicto/integridad 409; validación 422; readiness fallido 503. La UI conserva el mensaje legible y la referencia de diagnóstico.

VALIDATION_RUN_REQUIRED, VALIDATION_RUN_OUTDATED, TECHNICAL_VALIDATION_REQUIRED y ADMINISTRATIVE_REASON_REQUIRED protegen las excepciones. VERSION_CONFLICT evita sobrescritura por formularios obsoletos. UNSUPPORTED_FORMAT, INVALID_DATA y UPLOAD_TOO_LARGE explican rechazos de carga. DATASET_NAME_EXISTS distingue el nombre duplicado de un conflicto genérico.

Hardening objetivo: rate limiting, federación OIDC, sensibilidad/masking, política de retención, administración productiva de sesiones y pruebas de penetración. No deben asumirse por existir sesión/RBAC y administración local.

20. Configuración, operación y respaldo

La configuración es por entorno y defaults locales. `.env.example` es una plantilla sin secretos operativos. `.env`, `storage`, datos de usuario, `caches`, `logs`, resultados temporales y `backups` se excluyen de `Git` y del contexto `Docker` según `.gitignore/.dockerignore`.

Variable	Función / default
DATABASE_URL	PostgreSQL en Compose; SQLite local en ejecución directa.
TRACKVANCE_STORAGE_DIR / ROOT	Raíz interna; <code>/var/lib/trackvance</code> en Compose.
TRACKVANCE_WEB_ORIGIN	Origen esperado; debe coincidir con <code>WEB_PORT</code> .
COMPOSE_PROJECT_NAME / WEB_PORT	Instalación y puerto; no confundir volúmenes entre proyectos.
POSTGRES_DB/USER/PASSWORD	Acceso local a metadata; password sólo en entorno.
TRACKVANCE_SECRETS_DIR	Volumen de credenciales cifradas externas.
TRACKVANCE_SECRET_KEY_FILE	Clave maestra local en volumen independiente.
TRACKVANCE_DESTINATION_SECRETS_DIR	Volumen independiente de credenciales cifradas de destinos.
TRACKVANCE_DESTINATION_SECRET_KEY_FILE	Clave maestra independiente de secretos de destino.
TRACKVANCE_WORKER_LANE	DEFAULT o DELIVERY; selecciona la cola y heartbeat del proceso.
DEMO_ACCESS_ENABLED	Habilita únicamente el login demo explícito; garantiza la identidad y organización mínimas, no datos sintéticos.
DEMO_SEED_ENABLED	Habilita únicamente el seed idempotente de datasets, configuraciones, ejecuciones, hallazgos y excepciones demo.
MAX_UPLOAD_BYTES / TRACKVANCE_MAX_ROWS	10 MiB y 100.000 filas por defecto.
TRACKVANCE_WORKER_MEMORY_SOFT_BYTES	Presupuesto blando del planner.
TRACKVANCE_TEMP_MIN_FREE_BYTES	Reserva mínima de disco temporal.
TRACKVANCE_RUN_TIMEOUT_SECONDS	Valor del plan; no timeout duro certificado.

Para conservar el cierre seguro de instalaciones anteriores, si `DEMO_ACCESS_ENABLED` está ausente hereda el valor de `DEMO_SEED_ENABLED`. Al definir ambas variables explícitamente sus comportamientos son independientes.

`TRACKVANCE_ENV` aparece como etiqueta en la plantilla; no selecciona por sí sola una política de seguridad productiva. No hay `Settings Pydantic` ni `feature flags` persistidos actualmente.

`Readiness` comprueba `SQL`, `Alembic head` y escritura temporal. `doctor.py` comprueba `HTTP`, servicios, migraciones y heartbeat, sin imprimir `DSN` ni secretos. `verify_storage.py` snapshot calcula hashes de registros de negocio, valida todos los artifacts y verifica referencias/credenciales cuando corresponde; compare exige igualdad antes/después de un reinicio en reposo. Entre capturas no deben ejecutarse `login/exports/tests` que agregan auditoría legítima.

El respaldo `SQLite` usa una copia consistente, integra `WAL`, verifica hashes y restaura en destino nuevo con rutas reubicadas. Incluye credenciales/clave de fuentes y credenciales/clave de destinos; una referencia sin su secreto o clave invalida el respaldo. La restauración conserva esos stores con acceso restringido y mantiene compatibilidad con backups históricos anteriores a las capacidades correspondientes.

Respaldo Docker consistente

scripts/docker_state.py backup requiere proyecto explícito Trackvance, etiquetas Compose verificables y los seis volúmenes esperados. Detiene temporalmente web, scheduler y ambos workers, captura el estado consistente, detiene API y produce pg_dump custom de PostgreSQL. Copia trackvance_data, connection_credentials, connection_keys, delivery_credentials y delivery_keys con su inventario; no copia en caliente el data directory PostgreSQL. Finalmente restaura sólo los contenedores que estaban activos al comenzar.

El formato nativo 0.5.1 usa backup-manifest.json schema 2 y state.json schema 4. El verificador etiqueta la huella según la revisión real y el inventario congelado del runtime: 0007/state 2, 0008/state 3 o 0009/state 4; no según la versión del script copiado. El manifest registra consistencia quiesced, fecha, proyecto, Alembic, versión PostgreSQL, revisión Git, imágenes y hashes/tamaños de cada componente. state.json conserva la huella verificable de metadata, artifacts y referencias. verify comprueba manifest, componentes exactos, hashes, rutas y archivos; rechaza enlaces y archivos no declarados. El conjunto contiene la clave necesaria para descifrar las credenciales y debe guardarse con acceso restringido. .env se conserva separadamente y no se imprime ni se incluye automáticamente en el backup.

Reconocer la huella 0007 no amplía la topología del CLI de backup actual: exige seis volúmenes y dos workers. Para crear un respaldo sobre una instalación todavía en 0007 se emplean las herramientas archivadas de esa release. El restore actual sí acepta sus backups verificables; para runtime 0008 el tooling nuevo conserva la huella nativa state 3.

La transferencia de archivos usa streaming: el helper escribe el tar por stdout hacia un archivo abierto por el proceso del host; la restauración envía ese archivo por stdin. No carga el archivo completo en RAM ni necesita un bind mount del directorio del host o elevar el helper a root. En POSIX se crean directorios de backup 0700 y archivos 0600; los contenidos restaurados conservan los modos verificados. El helper sólo monta el volumen necesario y permanece sin red.

Cada componente Docker se copia primero con creación exclusiva (O_EXCL) a un staging privado, se sincroniza con fsync, se vuelve read-only y se revalida por tamaño, hash e inventario antes de restaurar o inspeccionar. Todos los consumidores usan ese staging. El backup SQLite copia a un destino nuevo y verifica tamaño/hash antes de abrir o mutar la base. Esto evita consumir bytes sustituidos por una carrera TOCTOU entre inventario y uso.

```
python scripts/docker_state.py backup --project trackvance-local --destination backups/local
python scripts/docker_state.py verify --source backups/local
```

Restauración en proyecto nuevo

restore exige nombre distinto del origen y ningún recurso preexistente. Reconstruye volúmenes nuevos, restaura el dump en una transacción, repone artifacts y ambos pares credenciales/clave, inicia API y valida Alembic y la igualdad de la huella. Sólo inicia el resto del sistema con --start; sin esa opción deja el destino detenido tras verificarlo. --smoke requiere --start y está reservado para un ensayo aislado con identidad demo disponible. La salida es un recibo de restauración con hash del manifest y estado de verificación, sin secretos.

```
python scripts/docker_state.py restore --source backups/local `
--target-project trackvance-restored --web-port 3200 --start
```

La instalación destino debe disponer del código/imágenes y configuración de entorno compatibles. Una restauración fallida detiene el proyecto nuevo para diagnóstico; no borra el origen ni lo reemplaza. La prueba E2E destruye sólo su entorno temporal, restaura y comprueba hashes y funcionalidad, incluidos secretos y reconexión a motores de prueba. La evidencia ejecutada se registra en aceptación.

La compatibilidad 0.4.1→0.5.0 se certificó desde la baseline exacta 92c58eae9a1ddef653c0c9c888cf7698ee1af3a: manifest 1/state 2/0007, 21 tablas y SHA canónico legacy idénticos; restore actual en 0008/24 tablas, tres tablas Delivery vacías y 19 jobs históricos en lane DEFAULT. State 2 no guardaba un hash estructural del catálogo, por lo que no se atribuye esa garantía histórica.

El drill nativo 0.5.0 restauró el formato actual en un proyecto nuevo después de destruir la fuente. Verificó siete artifacts, dos secretos, 127 relaciones y Alembic 0008; la credencial de destino restaurada permitió una entrega posterior COMMITTED de cuatro filas. Doctor, hashes, linaje, stores separados y búsqueda de fugas de secretos aprobaron.

El drill ampliado trackvance-recovery-src-stream-e20a -> trackvance-recovery-dst-stream-e20a terminó PASS utilizando la transferencia por streaming: eliminó el proyecto fuente antes de restaurar y verificó siete artifacts, un secreto, 124 relaciones y Alembic 0007_monitor_scheduling. La credencial PostgreSQL restaurada permitió conectar, refrescar a DatasetVersion v2 y ejecutar el mismo contrato: el nuevo Run fue APPROVED y el original permaneció REJECTED. Se conservaron hashes, linaje e

historia y pasaron doctor, smoke API y búsqueda de secretos en logs. El resultado saneado se versiona en docs/development/evidence/0.4.0/recovery.json.

La recuperación incluyó una excepción con responsable/SLA y un adjunto, además de una programación Sentinel, dos revisiones, una ocurrencia y 31 muestras de métricas. El caso conservó su timeline y quedó PENDING_VALIDATION con validación técnica VALIDATED tras la nueva ejecución; auto_resolve_enabled permaneció false. La programación siguió pausada y su ocurrencia, métricas y alertas históricas se conservaron. El drill verificó HTML HTTP 200; Playwright no se ejecutó dentro de este ensayo y su certificación es independiente. Su resultado schema 3 y los ensayos previos quedan como evidencia separada.

Reset deliberado

El reset destruye metadata, datasets, runs, artifacts y secretos del proyecto seleccionado. El script Windows soportado detecta COMPOSE_PROJECT_NAME y delega en plan-reset/reset de docker_state.py. El plan enumera nombres e IDs exactos de contenedores, volúmenes y redes; incluye hash y caducidad de 15 minutos. La ejecución exige el token literal de confirmación y vuelve a verificar que el inventario no cambió. Un proyecto ajeno a Trackvance, un plan alterado, vencido o con recursos cambiados se rechaza. .env se conserva por defecto. No se usa reset para actualizaciones o pruebas sobre la instalación principal.

Para actualizar: terminar runs, respaldar, reconstruir imágenes, aplicar Alembic al iniciar, esperar healthchecks y comparar almacenamiento. Nunca ejecutar down -v sobre la instalación del usuario como parte de un upgrade.

Como antecedente operativo de 0.4.0, la instalación principal se actualizó entonces a Alembic 0007 después de un backup integral verificado. API, worker, web y PostgreSQL quedaron saludables con restart=no; la comprobación conservó los registros previos. Esa actualización histórica no utilizó la instalación principal para pruebas destructivas y no se presenta como estado actual de 0.5.0.

20A. Benchmark reproducible y límites comprobados

scripts/tests/benchmark_cycle.py ejecuta un proyecto Docker aislado con archivos sintéticos, fuentes PostgreSQL/SQL Server reales, materialización canónica, Intake, ReconOps y Sentinel. Registra bytes, filas, duración, throughput, CPU, memoria, I/O, snapshots, plan y éxito/fallo; watchdog detiene su proyecto si supera recursos o tiempo. La progresión exige preflight y termina ante una restricción de recursos. Los tamaños no ejecutados se marcan como tales.

Antecedente de volumen 0.4.0; no certifica 0.5.1

El ciclo principal trackvance-bench-22852-894497 terminó PASS. Su fixture nominal de 100 MiB mide 106.194.531 bytes, 50.000 filas y cuatro columnas, con payload determinista variado SHAKE256_URLSAFE_V1. Docker Desktop 29.1.3 dispuso de 16 CPU y 16.326.524.928 bytes de memoria. La duración total fue 79,46 segundos. Las duraciones siguientes son tiempos de pared del runner, incluyendo transporte/espera cuando corresponde; no son microbenchmarks puros del motor.

Operación	Duración medida
Carga archivo origen / destino	1,577 s / 1,306 s.
Intake desde archivo	3,083 s; 16.218 filas/s.
ReconOps desde dos archivos	4,076 s.
Sentinel desde archivo	2,046 s; 24.437 filas/s.
Snapshot PostgreSQL / Intake asociado	2,363 s / 3,112 s.
Snapshot SQL Server / Intake asociado	1,345 s / 3,067 s.

El HWM de proceso fue 812.998.656 bytes en API y 1.051.635.712 en worker; los picos cgroup fueron 1.253.138.432 y 1.224.929.280 respectivamente. El muestreo agregado registró 2.697.222.681 bytes con nueve muestras y CPU máximo agregado de 122,41 %, donde 100 % equivale a un núcleo. Los contenedores no registraron OOM. El almacenamiento temporal tuvo pico observado de 185.340.131 bytes y el crecimiento de storage fue 670.790.434 bytes, con 112 muestras cada 0,2 s. Los muestreos no garantizan capturar todos los picos transitorios; se conservan junto con cgroup/HWM en el JSON.

Representatividad y progresión detenida

El payload variado tiene 2098 caracteres por fila. El archivo produjo Parquet canónico de 79.145.600 bytes. La fuente PostgreSQL leyó 106.144.494 bytes lógicos y produjo 55.099.067 bytes canónicos; SQL Server leyó 105.750.000 y produjo 55.107.251. Se conservaron hashes y linaje de estos snapshots. Las fuentes y el archivo usan generadores deterministas distintos; sus tamaños comprimidos no deben equipararse.

El ensayo anterior trackvance-bench-25400-6b857b también pasó con payload repetido y Parquet SQL de sólo 26.909 bytes; se conserva como antecedente de compresibilidad, no como resultado principal de presión de recursos. La medición variada mejora esa cobertura, pero sigue siendo un perfil de 50.000 filas y cuatro columnas. No establece un máximo universal de Polars ni certifica millones de claves, todas las formas de reglas o un dataset con 100 columnas. Otro perfil requiere otro ensayo reproducible.

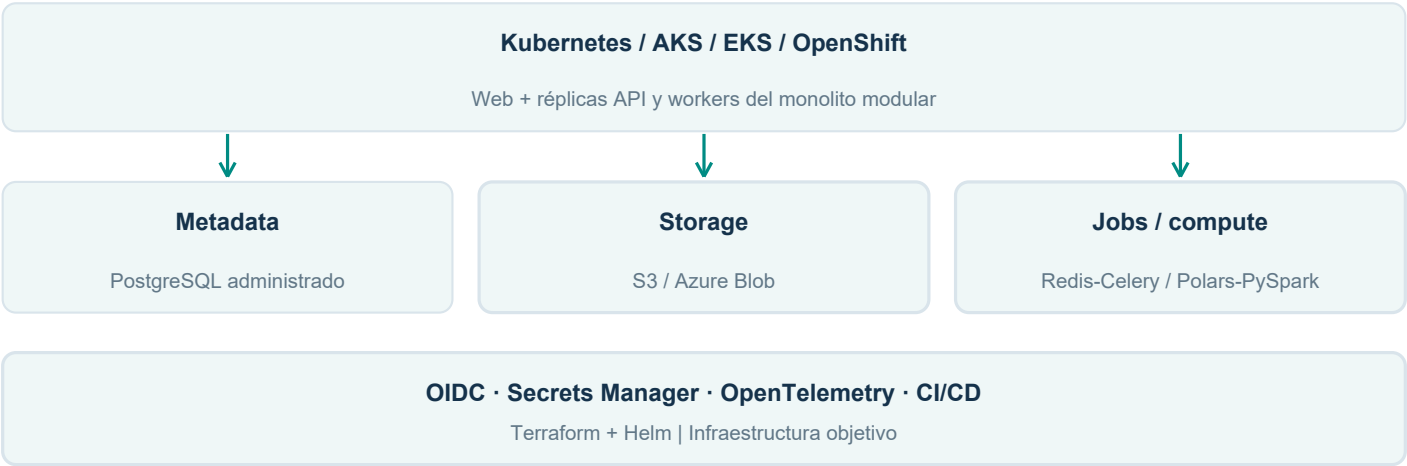
Se usaron overrides exclusivos del proyecto temporal: upload de 128 MiB, snapshot de 160 MiB, presupuesto blando del planner de 6 GiB y 1200 segundos en el plan. Los defaults de la instalación no se modificaron. El Recon estimó 5.097.337.488 bytes para ambos inputs; el watchdog no registró una violación.

500 MiB, 1 GiB, 2 GiB y 5 GiB aparecen NOT_RUN_RESOURCE_LIMIT porque la estimación Recon excedía el presupuesto local. No son fallos de ejecución medidos. El máximo comprobado de este perfil es el fixture indicado, con sus overrides y limitaciones. No se proclama completada una matriz de volumen que no se haya ejecutado.

Se conservan los límites operativos por defecto de 10 MiB/upload, 100.000 filas, 100 columnas y 64 MiB de snapshot SQL. Los warnings/rechazos de ExecutionPlanner siguen el presupuesto configurado de memoria/disco y ENGINE_UNAVAILABLE cuando solicita un engine no instalado. La evidencia no justifica elevarlos ni fijar un umbral universal de migración a PySpark. PySpark se recomienda como evaluación de producto ante cargas que superen el presupuesto local, pero su adaptador no se implementa en este ciclo.

21. Arquitectura final de producto

OBJETIVO. El producto final conserva los mismos módulos y contratos funcionales. Evoluciona la entrega de jobs, almacenamiento, fuentes/destinos, identidad, observabilidad e infraestructura. No divide Intake, ReconOps, Sentinel o Data Delivery en microservicios.



Capacidad	Adaptador / infraestructura objetivo
Despliegue	Docker y Kubernetes, AKS, EKS u OpenShift; Helm para releases.
Metadata	PostgreSQL administrado, backups y recuperación verificados.
Storage interno	S3 o Azure Blob mediante StorageProvider, locators, hashes y retención.
Fuentes	PostgreSQL y SQL Server implementados; S3, Azure Blob, APIs y otros motores futuros mediante DatasetSource.

Capacidad	Adaptador / infraestructura objetivo
Destinos	PostgreSQL y SQL Server implementados mediante DataSink; otros sinks, aprobación y políticas granulares futuros.
Jobs	Redis + Celery, reintentos/leases e idempotencia; entrega transaccional definida.
Compute	Polars para cargas estándar, PySpark para grandes volúmenes con paridad certificada.
Identidad	OAuth2/OIDC + SSO con Entra ID, Keycloak u otro IdP; RBAC/org scope propios.
Secretos	Key Vault, AWS Secrets Manager o Vault; referencias fuera de configs de negocio.
Observabilidad	OpenTelemetry, Prometheus, Grafana y logs centralizados.
Infraestructura	Terraform y configuración por ambiente, redes y permisos mínimos.
CI/CD	Quality gates, análisis de dependencias/SAST/secretos/imágenes, SBOM y despliegue controlado.

Pasos de evolución sin cambiar reglas

1. Introducir locator remoto y materialización acotada en un StorageProvider, migrando referencias y validando hashes históricos.
 2. Ampliar PostgreSQL/SQL Server con otros conectores DatasetSource/DataSink, límites, credenciales externas, snapshots y receipts reproducibles.
 3. Implementar JobQueue Redis/Celery con patrón transaccional/outbox o garantía equivalente; conservar Run y evidencia idempotentes.
 4. Añadir ExecutionEngine PySpark, capacidades del planner y pruebas de paridad antes de habilitarlo.
 5. Integrar IdP, secretos y observabilidad; aplicar tests de aislamiento y recuperación.
 6. Desplegar réplicas API/workers y certificar concurrencia, presión de recursos, backup/restore y upgrades.
- Los puertos actuales reducen el área de cambio, pero no certifican esos adapters. La transición requerirá código de infraestructura y migraciones; no deberá reescribir la semántica de reglas, resultados ni trazabilidad.

22. Calidad, pruebas y aceptación

La certificación local ejecuta pytest, Ruff, Mypy, ESLint, TypeScript, Vitest, build frontend, Playwright, smoke API, migraciones PostgreSQL y prueba de persistencia tras restart. El runner crea un proyecto Compose temporal con sus propios volúmenes y elimina únicamente esos datos al terminar.

Verificación	Resultado ejecutado
Corte y procedencia	25 septiembre 2026, rama feat/local-prototype. Código 8927ea00703ff090708c302d7755dae20a7687b9 sobre baseline e7838c3. Ejecuciones nuevas 0.5.1; ningún resultado 0.5.0 se hereda como PASS.
Backend local	1.091 pytest backend/scripts PASS en 65,97 s; dos avisos Starlette/AnyIO. Ruff PASS y Mypy PASS sobre 39 archivos fuente. Incluye regresiones de publicación de evidencia y backfill de arranque.
Frontend local	ESLint, TypeScript y build PASS. 154 Vitest en 17 archivos, 10,96 s. Siete Playwright con API interceptada sobre preview de producción PASS en 13,6 s; no equivalen a SQL real.
Bundle inicial	611.407 a 364.966 bytes; gzip 179.471 a 116.768 bytes (-34,94 %). Sin advertencia de chunk mayor de 500 kB; 19 chunks JavaScript y cuatro CSS. Carga diferida nativa, sin nueva biblioteca.
OpenAPI y esquema	Runtime 0.5.1: 83 paths, 71 schemas, 16 paths Delivery; JSON comparado exactamente por test. Única migración nueva 0009_delivery_reviews; 0001 a 0008 sin cambios.
Conexiones reales locales	96 comprobaciones PASS con PostgreSQL y SQL Server; 14/18 columnas temporales, refresh e historia, Intake, Recon, Sentinel y Delivery. Smoke API 84 PASS; Playwright 28 PASS y dos opt-in omitidos. Corrida anterior a los dos escenarios UI tardíos.

Verificación	Resultado ejecutado
Delivery real local	114 comprobaciones PASS, incluidas 22 de métricas PostgreSQL 16/18. Cuatro estrategias con ambos motores; Playwright real 1/1 PASS, restart y secretos PASS. Conteos desconocidos permanecen null/N/D.
Compose integral local	PASS: API 84/84 en 10,9 s, Playwright 24 PASS y seis opt-in omitidos en 2,0 min. Migraciones y restart exacto: 206 artifacts, 1.605 relaciones, state 4/0009. Corrida anterior a los dos escenarios UI tardíos; CI del código final se informa aparte.
Recovery y migraciones	PASS real: nueve artifacts, dos secretos, 164 relaciones, dos intentos y una revisión preservados. Reparación concurrente REPAIRED/ALREADY_VALID y repetición tras restore ALREADY_VALID; cuatro filas remotas invariantes. Roundtrip 0008/0009 conserva 24 tablas y ocho enlaces históricos.
Compatibilidad legacy	Nuevos drills PASS: backup auténtico 0.4.1 state 2/0007, siete artifacts y un secreto; runtime auténtico 0.5.0 state 3/0008, nueve artifacts y dos secretos. Ambos restauran a state 4/0009 con DTOs, bytes y proyección histórica exactos. Tooling nuevo sobre 0008 coincide con backup antiguo. Instalación principal intacta.
Benchmark general local	Smoke file-only PASS: 1.074.923 bytes, 1.000 filas, cuatro columnas; 39,4225 s; pico agregado 379.479.652 bytes, sin OOM. No certifica volumen ni Delivery.
Benchmark Delivery local	PostgreSQL 16 y SQL Server 2022: smoke 8/8 PASS, 1.074.923 bytes/1.000 filas/143,59 s; representativo 8/8 PASS, 8.917.809 bytes/20.000 filas/207,70 s. Pico agregado 2.050.125.461 bytes, sin OOM. Primer fixture incorrecto FAIL conservado, no contado como éxito.
Límites de volumen	100/500 MiB y 1/2/5 GiB: NOT_RUN_RESOURCE_LIMIT por límite de entrada vigente de 10 MiB, no por incapacidad de hardware demostrada. Límites del producto sin aumento. Una corrida por combinación; sin percentiles ni extrapolación productiva.
GitHub Actions	SUCCESS, ocho jobs: backend, frontend, compose-e2e, connections-e2e, delivery-e2e, backup-restore-e2e, benchmark-smoke y delivery-benchmark-smoke. Código 8927ea0; run 36194431770, workflow Trackvance local checks. Evidencia verificable: https://github.com/eddiean422/TrackvanceCore/actions/runs/36194431770
Navegador en CI final	Compose 26 PASS/seis opt-in omitidos, más clean-demo 1/1; Conexiones 30 PASS/dos omitidos; Delivery 1/1. Al deduplicar archivo/título parametrizado: 32 escenarios distintos con PASS, siete de API interceptada y 25 restantes. No se suman duplicados ni se convierten SKIP en PASS.
UNKNOWN real	NOT_RUN_NONDETERMINISTIC para pérdida exacta de confirmación remota post-commit. El fixture SIMULATED_UNKNOWN_NO_REMOTE_IO prueba revisión, persistencia e inmutabilidad; no esa ventana física de red.
Publicación PDF	El candidato se renderiza y revisa íntegramente antes de --publish. Número de páginas, inspección visual y hash idéntico de candidato/publicado se registran fuera del PDF en pdf-verification.json para evitar una referencia circular.

Certificación incremental 0.5.1

La evidencia de esta edición proviene de nuevas ejecuciones. Cada suite, motor, migración, benchmark y job CI tiene resultado propio; un escenario no ejecutado mantiene NOT_RUN con motivo. La prueba determinista de UNKNOWN no se presenta como reproducción física de una pérdida exacta de confirmación remota. La revisión visual del PDF se realiza después de incorporar los resultados finales.

Antecedente certificado 0.5.0 (no certifica 0.5.1)

Ningún éxito de 0.4.1 se hereda como certificación de 0.5.0. La aceptación debe cubrir suite backend/scripts, calidad estática, frontend, migración 0008, Compose/persistencia, Conexiones, Data Delivery con ambos motores, UI, estado UNKNOWN, secretos/linaje y backup/restore con entrega posterior. Resultado final: 869 pruebas Python y 128 Vitest aprobadas; calidad estática y build PASS; Playwright 20 aprobadas/5 opt-in omitidas; smoke 84/84, Conexiones 62/62 y Playwright focal 2/2, Delivery real 92/92 y Playwright Delivery 1/1; recovery y compatibilidad real 0.4.1→0.5.0 PASS. GitHub Actions: workflow 35875426350, commit 57c0319ae2f0ef27ae63eced8e354258efc0854d, siete jobs SUCCESS sin reintentos: backend, frontend, compose-e2e, connections-e2e, delivery-e2e, backup-restore-e2e y benchmark-smoke. La URL está en la tabla de resultados.

CI registró Python 869/869 en 50,08 s y frontend 128/128; Playwright Compose 19 aprobadas/6 opt-in omitidas más demo limpio 1/1, Conexiones 23 aprobadas/2 omitidas y Delivery focal 1/1. Estas ejecuciones no se suman como escenarios únicos; el cruce de nombres confirma 25 escenarios distintos con al menos un PASS. Los cuatro opt-in de fuentes SQL pasan en Conexiones, demo limpio en su paso separado y Delivery en su job. La compatibilidad con el baseline 0.4.1 se certificó localmente; no es un job adicional de CI. Los JSON saneados de las pruebas locales y sus límites están en docs/development/evidence/0.5.0. El commit posterior del PDF y del dato de Actions es un cierre documental del código de producto certificado.

Base certificada de 0.4.1

La tabla histórica de 0.4.1 fue la fuente explícita de esa revisión; ningún éxito se hereda de 0.4.0. En local aprobaron 533 tests backend/scripts, Ruff, Mypy sobre 33 archivos, ESLint, TypeScript, build y 119 tests Vitest en 13 archivos. El bundle principal fue de 550,78 kB / 164,59 kB gzip y conserva un aviso informativo de tamaño.

El ciclo Compose aislado aprobó doctor 7/7, smoke API 84/84, migraciones upgrade/downgrade/upgrade, persistencia tras restart y 19 escenarios Playwright; cinco opt-in se cubrieron en ciclos especializados. El ciclo real de Conexiones aprobó 62 comprobaciones PostgreSQL/SQL Server y 23 escenarios de navegador; el recorrido limpio separado aprobó 1/1, incluido logout, redirección a / e inicio visible. Son 24 escenarios Playwright distintos aprobados; las omisiones no se cuentan dos veces. No se añadió migración, endpoint ni DTO.

Base certificada de 0.4.0

La ejecución consolidada local aprobó 533 tests pytest en 41,66 s, con dos warnings de deprecación de las dependencias de pruebas. Incluye nueve regresiones nuevas del respaldo/restauración por streaming. Ruff pasó; Mypy comprobó 33 archivos. ESLint, TypeScript y build frontend pasaron; Vitest aprobó 109 tests en 12 archivos. El bundle principal compilado fue de 535,26 kB; su aviso de tamaño no se presenta como fallo de compilación.

La certificación de Conexiones completó 62 comprobaciones con PostgreSQL y SQL Server reales y 23 pruebas Playwright, omitiendo sólo el escenario clean-demo de activación explícita. Ese escenario pasó por separado: en total se probaron 24 escenarios de navegador distintos. El ciclo Compose aprobó 19 pruebas Playwright; sus cinco casos opt-in se cubrieron en las ejecuciones independientes y no se suman como escenarios nuevos.

El ciclo Compose también aprobó smoke API, doctor, migraciones PostgreSQL e integridad después del reinicio. La comparación conservó exactamente 21 tablas, 56 datasets, 70 versiones, 43 runs, 206 artifacts, 1562 relaciones y un adjunto. El drill de recuperación ampliado se describe en la sección 20 y el benchmark medido en 20A. Las pruebas se ejecutaron en proyectos y volúmenes aislados. El resultado de GitHub Actions se informa separadamente; estos éxitos locales no equivalen a un workflow remoto aprobado.

Escenarios de negocio conservados

Escenario	Resultado certificado por tests
Intake 120 filas	110 válidas, 10 con error, aceptación 91,67 %, REJECTED.
Detalle Intake	REQUIRED 4, UNIQUE 4 registros, POSITIVE 2.
Recon 120/110	MATCH 110, SOURCE_ONLY 6, DUPLICATE_SOURCE 4; otras categorías 0; match 91,67 %.
Sentinel baseline	110 filas, 9 checks, 0 fallos, salud 100 %.
Sentinel v2	90 filas, 3 nulls monitorizados, cambio 18,18 %, 4 fallos, salud 55,56 %.
Sentinel v3	90 filas, source_system ausente, cambio 0 %, 1 fallo, salud 88,89 %.
Excepciones	Bloqueo sin validación; Run posterior correcto habilita RESOLVED; cierre administrativo separado.

La base 0.4.0 incluye cinco formatos, hoja Excel, delimitador TXT, JSON anidado simple, esquema embebido Parquet, overrides, identificadores/Todos, selectores de reglas, navegación, dashboard, VALUE_MISMATCH, not_future, schema drift, XLSX/MIME/filename/formula injection, RBAC/CSRF y lineage. La cobertura incremental 0.4.1 comprueba el selector único de identificadores, catálogo de responsables, labels/ayudas/orden/preview de transforms, preview de claves simples y compuestas, multiselect de Sentinel por esquema y regreso al inicio tras logout.

El workflow ci.yml declara checks backend/frontend, migraciones y ciclos Compose/conexiones/delivery/recuperación con entornos temporales y evidencia de navegador. Los jobs realmente ejecutados y su resultado se registran en la tabla de validación. Conexiones usa usuarios SELECT; Delivery usa destinos PostgreSQL/SQL Server aislados. Ambos buscan fugas de secretos y no

apuntan a la instalación principal. La ejecución local no equivale a una ejecución remota de GitHub Actions. Security scanning/SBOM y gates de producción permanecen en el objetivo final.

La revisión 0.4.1 quedó certificada en GitHub Actions por el workflow 35638817011, commit 38a8c7a3dd14ecc1c6b730f2a4a98e419707281e: seis de seis jobs terminaron SUCCESS (backend, frontend, connections-e2e, compose-e2e, benchmark-smoke y backup-restore-e2e). Este resultado corresponde al mismo código de producto y fuentes documentales usados para la publicación; la incorporación posterior del PDF y del dato de CI es un cierre documental.

Incidencia CI identificada y corrección

El primer workflow 35488396043 aprobó cinco de seis jobs, pero backup falló en Linux. El helper se ejecutaba como usuario trackvance e intentaba escribir sobre un bind del host con permisos 0755 de otro propietario; el sistema devolvió PermissionError 13. La causa se reprodujo y se corrigió transportando el tar por stdout/stdin con archivos privados abiertos en el host. No se relajaron permisos del host ni se ejecutó el helper como root. Pasaron 48 pruebas focalizadas, la suite completa de 533 tests y el drill real de recuperación por streaming.

La corrección quedó certificada en GitHub Actions: workflow 35488848150, commit a03d87ecb051d88abfe57111ae90853f9dcbcf66, resultado SUCCESS en seis de seis jobs: backend, frontend, compose-e2e, connections-e2e, backup-restore-e2e y benchmark-smoke. El job de backup verificó siete artifacts, un secreto y 124 relaciones. Backend aprobó 533 tests, Ruff, Mypy y migraciones; frontend aprobó sus 109 tests, lint, tipos y build. Los escenarios opt-in de navegador se cubrieron en los jobs correspondientes. El benchmark smoke valida el runner con carga pequeña; el volumen local de 100 MiB conserva su certificación independiente. La URL verificable del workflow figura en la tabla de resultados; el fallo inicial permanece documentado como incidencia corregida.

Pendientes explícitos

PySpark, conectores/fuentes/sinks adicionales, scheduler y RBAC granular de Delivery, rotación automática de claves maestras, Redis/Celery productivo, OIDC/SSO, Kubernetes/Helm/Terraform, observabilidad distribuida y masking/sensibilidad permanecen fuera de este ciclo. La adquisición asíncrona de grandes volúmenes, timeout duro y agrupación multi-finding no se simulan. Los volúmenes no ejecutados, fallos o gates sin certificar se identifican explícitamente en validación y benchmarks; no se reclasifican como éxitos.

23. Decisiones vigentes y control documental

Decisión	Referencia del repositorio
Monolito modular local y dos modos de arranque	docs/adr/0001-local-prototype.md, antecedente 0.1.
Evidencia v2, actores, artefactos y compatibilidad	docs/adr/0002-evidence-and-artifacts.md.
Semántica observada y reglas declarativas	docs/adr/0003-rule-semantics.md.
Lectores y frontera común de calidad	docs/adr/0004-dataset-readers.md.
Resolución de excepciones validada técnicamente	docs/adr/0005-technical-validation-of-exceptions.md.
Puertos de storage, fuentes, ejecución y cola	docs/adr/0006-architecture-ports.md.
Conexiones, secretos, snapshots y frontera Data Delivery	docs/adr/0007-external-connections.md.
Reglas avanzadas, condiciones, referencias e identidad	docs/adr/0008-advanced-intake-rules.md.
ReconOps configurable, transforms, nulls y agregaciones	docs/adr/0009-configurable-reconciliation.md.
Scheduler Sentinel local y series compatibles	docs/adr/0010-local-sentinel-scheduler.md.
Asignación, SLA, adjuntos y resolución automática	docs/adr/0011-advanced-exception-workflow.md.
Administración local de usuarios y sesiones	docs/adr/0012-local-identity-administration.md.
Backup, restore y reset verificables	docs/adr/0013-local-backup-restore.md.
Benchmark y política de escalado con evidencia	docs/adr/0014-volume-benchmark-policy.md.
Data Delivery, transacciones, UNKNOWN, lanes y secretos de destino	docs/adr/0015-data-delivery.md.

Precedencia y cambios de esta revisión

Esta revisión sustituye las afirmaciones de implementación de la edición inicial de 52 páginas. La edición inicial se conserva archivada con su hash, como antecedente. Los diagramas aspiracionales de worker-spark, tablas de organizaciones/roles, librerías no instaladas, exports CSV principales y endpoints futuros ya no se presentan como realidad operativa.

Se actualizan arquitectura local/final, esquema real de 25 tablas, contrato HTTP, cinco lectores, Conexiones PostgreSQL/SQL Server, DataSink y Data Delivery SQL, SecretStores separados, lanes/workers, UNKNOWN, backup, UI y evidencia. Se preservan snapshots, profiling, reglas avanzadas, ReconOps, Sentinel, excepciones, usuarios/RBAC, Parquet canónico, manifests v2, Excel, cockpit, auditoría, persistencia Docker y arranque manual.

El código y OpenAPI determinan el contrato ejecutable. Un cambio posterior de semántica requiere ADR, pruebas, actualización de esta fuente y del PDF; un cambio de base de datos requiere Alembic nuevo. Las versiones de configs/manifests/métricas se gestionan separadas de la versión del producto.

Registro

Fecha	Revisión	Descripción
10-09-2026	Especificación v1.1 inicial	Diseño y objetivo de producto.
16-09-2026	v1.1, implementación 0.3.0	Consolidación del estado real, fronteras de infraestructura y validación local.
19-09-2026	v1.1, evolución Conexiones	PostgreSQL, SQL Server, secretos cifrados, snapshots y pruebas con motores reales.
19-09-2026	v1.1, implementación 0.4.0	Maduración funcional del roadmap local; resultados y límites según evidencia de esta revisión.
21-09-2026	v1.1, implementación 0.4.1	Configuración guiada por esquema, previews informativos, catálogo de responsables y cierre de sesión corregido; contratos del motor sin cambios.
23-09-2026	v1.1, implementación 0.5.0	Data Delivery PostgreSQL/SQL Server, destinos/configuraciones versionados, lanes, UNKNOWN, exactitud de tipos, locks/guardias transaccionales, secretos/evidencia/backup compatible y UI; validación local cerrada y CI informado por separado.
25-09-2026	v1.1, implementación 0.5.1	Ocho endurecimientos focales, capítulo Delivery A-Z, nueva certificación y benchmark propio; límites y resultados según evidencia 0.5.1.

La fuente editable y el generador portable acompañan al PDF en docs/specification; la copia oficial se conserva en Documentación. README, arquitectura, catálogo, ADRs, OpenAPI y documentación de operación complementan esta especificación con comandos y contratos de detalle. OpenAPI 0.5.1, el informe validation_results_0.5.1.json y esta fuente son entradas explícitas del generador. Publicar exige candidato y revisión visual completa; la tabla de validación conserva resultados locales y CI separados. Los archivos de resultados 0.5.0 y anteriores permanecen como antecedentes, nunca como certificación 0.5.1.

24. Cambios funcionales y técnicos 0.5.1

1. Reparación local de evidencia COMMITTED

Antes: el worker preservaba COMMITTED y marcaba PENDING_REPAIR, pero el operador no tenía una acción segura para completar receipt/manifest. Riesgo: confundir evidencia ausente con entrega fallida y volver a escribir.

Solución funcional: "Reparar evidencia" conserva el commit. Solución técnica: validación local estricta, bloqueo de Run, artifacts deterministas, publicación sin reemplazo y reutilización de enlaces. Archivos: delivery_service.py, delivery_api.py, delivery_schemas.py, DeliveryOperations.tsx y tests operacionales. Modelo: no añade tabla para reparar. API: POST repair-evidence con runs:execute.

Compatibilidad: conserva bytes 0.5.0 y no altera intentos ni migraciones anteriores. Pruebas: negar DataSink/SecretStore, repetición/concurrencia, parcial/ausente, tampering, organización, permisos y restore. Se cubre también la ventana normal COMMITTED anterior a receipt/manifest: UI y runners esperan publicación con límite, sólo por GET y sin ocultar PENDING_REPAIR. Límite: no puede inventar evidencia independiente ausente ni corregir bytes corruptos sobrescribiendo su hash.

2. Revisión operacional de UNKNOWN

Antes: UNKNOWN expresaba correctamente la incertidumbre, pero carecía de un historial estructurado de verificación externa. Riesgo: sobrescribir el intento original o esconder la conclusión en un comentario no consultable.

Solución funcional: observaciones append-only con tres outcomes, nota y revisor. Solución técnica: DeliveryOperationalReview y 0009_delivery_reviews, FKs/CHECK, scope y fecha validada; el worker deja intacto un attempt ya UNKNOWN. Archivos: models.py, migration 0009, delivery_service/api/schemas.py, worker.py y UI.

API: GET/POST reviews. UI: formulario e historial con advertencia de no replay. Compatibilidad: ninguna fila previa cambia. Pruebas: actor/fecha/organización, outcomes, historial, intento inmutable, sin Job nuevo, upgrade/downgrade y recovery. Límite: una afirmación humana no es confirmación automática ni aprobación dual.

3. Semántica uniforme de métricas

Antes: algunas descripciones trataban rows_written como filas físicas y podían dar a null apariencia de cero. Riesgo: inferir una población remota incorrecta, especialmente con UPSERT, triggers o filas preexistentes.

Solución funcional: mostrar filas enviadas y N/D explícito. Solución técnica: delivery_metrics.py centraliza metric_semantics versión 1; servicio, DTO, receipt, manifest y UI conservan null. Run nuevo persiste tiempos separados de preflight y escritura. Modelo: sin nuevas columnas ni reinterpretación histórica.

Pruebas: valores conocidos/desconocidos/cero, recibos históricos sin metadata aditiva y formato UI. Archivos: data_sinks.py, delivery_metrics.py, delivery_service.py y feature delivery. Límite: bytes preparados no son tráfico de red y filas enviadas no equivalen a inventario físico post-trigger.

4. Desglose UPSERT PostgreSQL por capacidades

Antes: el conteo diferenciado no podía garantizarse en todas las versiones. Riesgo: usar un SELECT previo sujeto a carreras o un detalle interno como xmax.

Solución: PostgreSQL 18 RETURNING OLD/NEW, preservando ON CONFLICT, locks y constraint nombrada; PostgreSQL 16 conserva null cuando no sabe. No modifica el servidor metadata ni exige actualización de destinos. Archivos: data_sinks.py, delivery_metrics_probe.py, delivery_cycle.py y overlay de destinos de pruebas.

API/UI reciben enteros o null existentes; no hay migración. Pruebas reales: inserción, actualización, mezcla, clave compuesta, vacío, rollback, drift, supresión por trigger y registro OLD totalmente null. Límite: acciones reportadas no incluyen un censo de efectos secundarios del receptor.

5. Regresión temporal exhaustiva

Antes: existía la política conservadora, con cobertura parcial de precisiones y tránsito entre módulos. Riesgo: que un cambio en driver/lector pierda fracción, invente zona o convierta un texto sólo porque parece una fecha.

Solución: matrices de precisión PostgreSQL y SQL Server, casos null/offset y valores exactos, refresh e historia, Intake/Sentinel/Recon y entrega. Archivos: test_dataset_sources.py, test_data_sinks.py, connections_cycle.py y fixtures. No cambia modelo, API ni política temporal como parte de la regresión.

Compatibilidad: snapshots anteriores permanecen inmutables. La UI conserva su tipo lógico y no ofrece coerción prohibida en mapping. Pruebas reales y unitarias se registran separadas. Límite: TIMESTAMP llega hasta microsegundos; el séptimo dígito o ausencia de zona se conserva como STRING. La adquisición SQLServer estilo127 puede normalizar a UTC; no se promete preservar la forma numérica original del offset anterior a materializar el snapshot.

6. Linaje canónico sin migración histórica

Antes: documentación confundía DELIVERED_TO con el tipo de entidad DELIVERY_DESTINATION_VERSION. Riesgo: lectores/documentación dibujando un grafo distinto del persistente.

Solución: tabla y diagrama exactos de source_type, relation y target_type; se mantienen DELIVERY_INPUT, DELIVERED_TO, DELIVERY_RECEIPT, EVIDENCE_OF y RUN_OUTPUT. Archivos: especificación, ADR0015, API_CONTRACT, services.py y pruebas de linaje. Modelo/API: ningún rename de datos ni backfill destructivo.

La reparación reconstruye los mismos enlaces faltantes y no inventa una relación nueva. El drill detectó que el backfill legacy de arranque añadía RUN_INPUT a Runs Delivery; ahora los excluye, sin borrar vínculos históricos preexistentes. Pruebas contrastan

triples, ausencia de duplicados y preservación exacta tras reinicio/restore. Límite: este grafo describe evidencia interna; no representa linaje a nivel fila en el sistema receptor.

7. Benchmark exclusivo de Delivery

Antes: el benchmark general cubría adquisición/calidad, no aislaba estrategias de salida. Riesgo: usar su throughput como si midiera Delivery.

Solución: runner dedicado, fixture variado determinista, ocho combinaciones motor/estrategia, smoke y representativo acotado, watchdog y recursos observados. Archivos: `delivery_benchmark_cycle.py`, `overlay delivery-benchmark`, `tests`, `workflow CI`, informe y evidencia JSON 0.5.1. API: utiliza contratos públicos; modelo/UI de producto no cambian por el benchmark.

Pruebas: guard de proyecto, límites sin aumento, generación determinista, mediciones reales y limpieza. Compatibilidad: no usa main ni datos del usuario. Límite: `NOT_RUN_RESOURCE_LIMIT` se conserva; no se extrapola a producción ni se afirma una precisión temporal/espacial mayor que la del muestreo.

8. División del frontend en módulos de carga diferida

Antes: rutas pesadas compartían un bundle inicial de 611,41 kB. Riesgo: coste de arranque y advertencia de tamaño, aunque la pantalla solicitada fuera pequeña.

Solución: `React.lazy/Suspense`, imports dinámicos y frontera de error; separación del detalle Delivery del builder. Archivos: `App.tsx`, `RouteContent.tsx`, `Runs.tsx`, `DeliveryRunDetail.tsx` y pruebas. Modelo/API: sin cambio de contratos por splitting. Compatibilidad: rutas, sesión, permisos, callbacks y navegación directa conservados.

Pruebas: Vitest incluye una promesa lazy rechazada para el fallo de carga; Playwright intercepta la API para rutas/deep links y se ejecuta por separado con backend real. No se atribuye al navegador una inyección de fallo de red del chunk. Build medido antes/después: bundle inicial nuevo de 364,97 kB; la suma de todos los chunks se informa por separado. Límite: la primera apertura de una feature puede requerir su descarga; una caché o red defectuosa debe mostrar error recuperable, no una pantalla vacía.